

DD: ILM-CFG-01 — Customer Service Module

Developer implementation guide: DD_ILM-CFG-01-Customer_Service-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_ILM-CFG-01-Customer_Service-v1.0.md.

Verdict: ■ New | **Wave:** 2 | **Group I:** Foundations | **Version:** v1.0 | **Module:** Customer Service The system of record for the Wananchi customer master. Owns the Customer and Account entities of the workshop's 3-tier CAS hierarchy. The third tier — the Subscription / service-instance — is owned by SUB-LM-01 and looked up cross-module via (customer_id, homepass_id). KYC lifecycle, account status + sub-status registry, contact methods, customer notes, preferences, search, and interaction history. The module is the read-after-anything-customer-related authority for every other module in SOPHIX. Retires customer_stub in FUL-02 framework and WO module framework. The module is operator-agnostic at the code level: WIK, WUG, WTZ, YASSN all use the same schema, the same REST API, the same Kafka event surface. Per-operator differences (KYC document types, status enums, language) live in operator-config rows.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/09_catalogs/DD_ILM-CFG-01-Customer_Service-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- DELETE /api/customer-accounts/{id}/flags/{flag_code}
- DELETE /api/customer-accounts/{id}/subscription
- DELETE /api/customers/{id}
- GET /api/customer-account-flag-catalog?operatorCode=WIK
- GET /api/customer-accounts/search?key=flag&flagCode=NPD&value=true
- GET /api/customer-accounts/search?key=payment_account_number&value=<ref>

- GET /api/customer-accounts/{id}
- GET /api/customer-accounts/{id}/flags
- GET /api/customer-accounts/{id}/flags/{flag_code}
- GET /api/customer-accounts/{id}/notification-pref
- GET /api/customer-accounts?customerId=...
- GET /api/customers/search
- GET /api/customers/search?key=msisdn&value=%2B254712345678&limit=10
- GET /api/customers/{id}
- GET /api/customers/{id}/contact-methods
- GET /api/customers/{id}/interactions?limit=20
- GET /api/customers/{id}/notes?limit=20&kind=...
- GET /api/customers/{id}/notification-prefs
- GET /api/subscriptions/subs_01HX_JANE_TRIPLE_PLAY
- GET /api/subscriptions/{subscription_id}
- GET /api/work-orders/count?customerId=...&statusNotIn=COMPLETED, CANCELLED
- PATCH /api/contact-methods/{id}
- PATCH /api/customer-accounts/{id}
- PATCH /api/customers/cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6
- PATCH /api/customers/{id}
- PATCH /api/notes/{id}/supersede
- POST /api/contact-methods/{id}/verify
- POST /api/customer-accounts
- POST /api/customers
- POST /api/customers/cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6/kyc/final-approve
- POST /api/customers/cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6/kyc/l1-approve
- POST /api/customers/{customer_id}/kyc/approvals
- POST /api/customers/{id}/contact-methods
- POST /api/customers/{id}/interactions
- POST /api/customers/{id}/kyc/final-approve
- POST /api/customers/{id}/kyc/l1-approve
- POST /api/customers/{id}/kyc/reject
- POST /api/customers/{id}/notes
- POST /engine-rest/external-task/fetchAndLock
- PUT /api/customer-accounts/acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7/sub-status
- PUT /api/customer-accounts/acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7/subscription
- PUT /api/customer-accounts/acct_01HZP9Q8R7S6T5U4V3W2X1Y0Za/flags/FRAUD_SUSPECTED
- PUT /api/customer-accounts/{id}/flags/{flag_code}
- PUT /api/customer-accounts/{id}/notification-pref
- PUT /api/customer-accounts/{id}/sub-status
- PUT /api/customer-accounts/{id}/subscription

4.2 Tables

- customer
- customer_account
- customer_account_flag
- customer_account_flag_catalog
- customer_account_notification_pref
- customer_account_status_history
- customer_contact_method
- customer_interaction
- customer_kyc_approval
- customer_kyc_document
- customer_note
- customer_sub_status_catalog

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- await-kyc-approval
- final-approve
- flag-evaluator-wik-v1
- flag-evaluator-yassn-v1
- kenya-kyc-leaders
- kenya-l1-kyc
- no-disturb
- sophix-superadmin

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- SubscriptionCreated
- SubscriptionPaused
- SubscriptionResumed
- SubscriptionTerminated
- WorkOrderCancelled
- WorkOrderCreated
- WorkOrderFinalized
- WorkOrderPhaseFinalized

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 2 | **Group I:** Foundations | **Version:** v1.0 | **Module:** Customer Service

The system of record for the Wananchi customer master. Owns the Customer and Account entities of the workshop's 3-tier CAS hierarchy. The third tier — the Subscription / service-instance — is owned by SUB-LM-01 and looked up cross-module via (customer_id, homepass_id). KYC lifecycle, account status + sub-status registry, contact methods, customer notes, preferences, search, and interaction history. The module is the read-after-anything-customer-related authority for every other module in SOPHIX. Retires customer_stub in FUL-02 framework and WO module framework.

The module is operator-agnostic at the code level: WIK, WUG, WTZ, YASSN all use the same schema, the same REST API, the same Kafka event surface. Per-operator differences (KYC document types, status enums, language) live in operator-config rows.

1. What this module is

Every internal team in Wananchi — CSR, dispatcher, technician, finance, sales agent — opens a customer record before any action. Workshop quote: *"The single pane of glass any internal agent opens before any action on the customer's account."* This module owns that record.

Customer is identity; Account is operations

The module enforces a strict separation between two layers:

- **Customer** — the legal-identity entity Wananchi recognizes: a person (Residential) or a registered business (Commercial). Holds name, type, identification documents, date of birth, primary contact channels (msisdn / email — the **legal-known** channels), preferred language, KYC state. Contact methods, notes, and interactions also hang from the Customer because they describe communication with the legal entity. **Nothing operational lives here.**
- **Account** — the operational anchor for everything Wananchi does for the customer. Each Account corresponds to one physical service deployment (one HomePass) and is linked 1:1 to one Subscription. Status + sub-status, service-class tier, attention banner, flags (NPD, churn risk, fraud, ...), notification preferences, payment account number, billing cycle dates, account manager, franchise code — all live on `customer_account`. **Every other module anchors at the Account, not the Customer:**

Concern	Where anchored
Work Orders (WO module)	Account (accountId filter on WO queries)
Billing & Charging (BIL-01)	Account (via the Account's subscription_id, 1:1)
Invoicing (BIL-02-GEN-01)	Account (one Invoice per Account per cycle)
Payments + reconciliation (BIL-05)	Account (resolved via payment_account_number)
Dunning (BIL-04)	Account (each Account dunned independently)
Provisioning (FUL-03)	Account (via the Account's Subscription)
Notification dispatch (DD_NOT-01)	Account (account-scoped notification preferences)
Status & lifecycle	Account
Flag system (NPD, etc.)	Account
Service class	Account
Attention banner	Account

A customer with multiple accounts (e.g., one at home, one at a holiday cottage) has each Account managed completely independently — separate status, separate flags, separate invoices, separate dunning, separate notifications. The Customer entity ties them together for legal identity only.

Capability areas

The module is structured around six capability areas, each backed by data and APIs:

1. **Master record** — the Customer entity (legal identity only — name, type, IDs, KYC, contact channels, language). The Account entity (per-deployment, one Account per physical service location per workshop rule). Subscription instances per package live in SUB-LM-01 — this DD references them via cross-module queries, does not own them.
2. **Status lifecycle** — Account-level. Main statuses ACTIVE / INACTIVE + a rich catalog of sub-statuses (active, complementary discount, DND active, seasonal disconnect, VIP, staff account, inactive-pending-customer, churned, etc.). Sub-statuses are operator-configurable per workshop's cloned-from-main pattern. Service-class tier (VIP / Platinum / Gold / STAFF) is informational, no billing impact.
3. **Account flag system** (§3.5) — operator-extensible per-Account flags (NPD, churn risk, fraud suspected, high-value, loyalty tier, ...). Evaluated daily by a Drools worker per operator. Workshop's NPD boolean flag becomes one entry in this system.
4. **KYC** — capture, two-level approval flow (L1 supervisor → Team Leader), IDs accepted (National ID, Passport, Business Registration), document references. Captured on the Customer (legal identity), but the activation gate is per-Account: an Account cannot be activated until its Customer's KYC reaches APPROVED. Future IPRS integration is out of scope for v1.0.
5. **Contacts + notes + interactions + preferences** — primary phone (MSISDN, required, enforced at every contact-update operation per workshop rule), email, alternate contacts, all on the Customer. Notes + interactions on the Customer with optional account_id scope (a CSR call about "Jane's Diani account billing" is account-scoped; a customer-relationship note like "deceased" is customer-scoped). Notification preferences (§4.4) are **account-scoped, not customer-scoped** — per-Account channel + topic preferences.

6. **Search** — by Account #, MSISDN, name, ID, address (with the workshop's required result columns: Address, Last Name, First Name, M St, E Stat, City, Pending WO, Type, Account Name). Also by flag (§7.8) for BIL-04 dunning and Finance reporting.

The module is exclusively a customer service authority — it does not own billing data (transaction history is read from billing modules), it does not own work orders (read from WO module), it does not own subscription state (read from SUB-LM-01). It owns identity (on Customer) + operational anchors (on Account). Other modules consume via REST + subscribe to Kafka events for status changes.

What this module does NOT do

- **Doesn't own billing or invoicing.** Transaction History tab on the workshop's "single pane of glass" reads from BIL-01 (Charging Engine) + BIL-02-GEN-01 (Invoice Generation) + BIL-05 (Wallet); ILM-CFG-01 just hands over `customer_id` (or `account_id`) and surfaces the join. Adjustments, refunds, billing-cycle changes — all live in BIL. The Account-to-Invoice relationship is documented in §11.4 BIL-* sibling entry.
- **Doesn't own work orders.** Pending WOs, Completed WOs grid, WO Notes — all read from WO module (DD_WO-01) via `customer_id` filter. ILM-CFG-01 provides the customer context; WO module owns the WO data.
- **Doesn't own subscription state.** Active/Suspended/Terminated subscriptions live in SUB-LM-01. ILM-CFG-01's status field is the *customer-level* lifecycle (ACTIVE vs INACTIVE + sub-statuses); per-subscription state is separate.
- **Doesn't compute dunning.** BIL-04 owns the dunning state machine. When a customer is dunned to SUSPENDED, BIL-04 emits an event and ILM-CFG-01 updates the customer's `sub_status` to reflect "inactive pending customer" or similar; this is event-driven consumption of dunning side-effects, not authority over dunning logic.
- **Doesn't initiate KYC document scanning or OCR.** v1.0 captures references to documents stored elsewhere (`id_document_ref`); future IPRS integration is out of scope. ILM-CFG-01 holds the audit trail of who approved what when.
- **Doesn't own customer bulk import** (workshop: not used in Kenya).
- **Doesn't own credit limit** (workshop: postpaid feature; Wananchi runs prepaid).
- **Doesn't own multi-location account support** (workshop: in Kenya, each location = separate Account; the multi-Account-per-Customer model handles this naturally).
- **Doesn't own document management** (KYC + contract live in Agent App; install photos with contractor's local FTP; future unified DM module is "good-to-have" per workshop).
- **Doesn't compute SLA** (workshop: SLA evaluation lives outside the platform today; future FUL-OPS-SLA may bring inside).
- **Doesn't own the customer risk profile or scoring** (workshop: not used in Kenya). v1.0 supports operator-extensible per-Account flags via §3.5 (NPD, churn-risk, fraud-suspected, etc.), evaluated by a daily Drools worker. The workshop's NPD boolean flag becomes one entry in this flag system, scoped to the Account.

Regional flexibility

The Customer Management workshop (deep-dive 01, 2026-04-27 / 2026-04-28) is a **Kenya** source. The schema and APIs here support Kenya's actual workflow — but the module is designed to deploy across all four Wananchi-Group countries (WIK, WUG, WTZ, YASSN) without code change. Regional variation is supported via:

- **Operator-scoped catalogs.** `customer_sub_status_catalog`, KYC document-type lists, contact-channel types, notification preferences — all keyed on `operator_code`. KE's sub-status list is the v1.0 seed; UG/TZ/SN seeds will be added at their respective deployment time without affecting the schema.
- **Multi-slot identifications.** `identification_type_1` + `identification_number_1` + `identification_type_2` + `identification_number_2` support regions where customers carry two IDs concurrently (e.g., ECOWAS card holds both a card-ID and a national-ID, common in Senegal/YASSN); KE typically uses `_1` only with `_2` NULL.
- **Optional fields.** Most workflow-specific fields (`franchise_code`, `account_manager_id`, `kyc_documents_jsonb`, identification slots) are NULL-able. Operators that don't use a concept simply leave it NULL; operators that do, populate it.
- **Multi-axis classification.** `service_class_1/2/3` and three classification slots support deployments that need multi-dimensional segmentation. KE v1.0 uses `_1` only.
- **Operator-extensible flag system.** `customer_account_flag_catalog` is operator-scoped — each operator defines its own flag types (NPD, churn risk, fraud suspected, payment delay, loyalty tier, etc.) with its own Drools rules. The KE seed (NPD + CHURN_RISK + FRAUD_SUSPECTED + PAYMENT_DELAY_FREQUENT + LOYALTY_TIER +

HIGH_VALUE) is v1.0; future operators add their own catalog rows without schema change.

- **Independent payment reference.** `customer_account.payment_account_number` is a separately-stored, mutable field — distinct from the immutable internal `account_number`. Operators with a custom gateway-reference scheme (KE M-Pesa Paybill, TZ Vodacom M-Pesa, SN/YASSN Orange Money or Wave) supply the reference per their gateway's format. Operator-config decides whether it's auto-populated from `account_number` at creation or supplied independently.
- **Per-operator BPMNs and Drools** (for the workflows that consume ILM events — FUL-02, WO module). The customer-service module just emits events; downstream operators decide what to do with them.

The Kenya workshop is the v1.0 seed, not the limit. Future workshop findings from WUG/WTZ/YASSN may reveal new identification types, status sub-statuses, KYC requirements, or contact channels — those land as catalog additions and per-operator config, never as schema migrations.

Glossary

Term	Meaning
Customer	The legal-identity entity Wananchi recognizes — a person (RES) or registered business (COM). Holds name, type, identifications, KYC, contact methods, language preference. Nothing operational lives here. All operational concerns (status, billing, work orders, invoices, payments, flags) anchor on the Account, not the Customer.
Account	The operational anchor for every internal workflow. Per-deployment (workshop rule: "1 account = 1 deployment"). Linked 1:1 to one Subscription. Multi-location customers hold multiple Accounts. Carries: status + sub_status, service-class tier, attention banner, flags (NPD, churn risk, ...), notification preferences, payment account number, account manager, franchise code, install/disconnect dates. The Account number is the only operational key used across all internal teams (workshop: "Account # is the only operational key across all sections and teams"). Work orders, invoices, payments, dunning — all anchored here.
Subscription	The service-instance entity, owned by SUB-LM-01. Workshop calls this tier "Subscriber" — V3 calls it "Subscription." From ILM-CFG-01's perspective, each Account is linked 1:1 to one Subscription via <code>customer_account.subscription_id</code> . A multi-service customer (Internet + TV + Voice) holds a single Subscription whose Package contains multiple Services; the multi-service decomposition lives in PLM-CFG-01 + SIP-01, not in additional ILM/SUB-LM rows.
3-tier CAS model	The workshop-named hierarchy: Customer / Account / Subscription. ILM owns Customer + Account. The Subscription tier lives in SUB-LM-01. The "CAS" naming is a workshop convention preserved here.
Main status	INACTIVE or ACTIVE only (workshop: "Base statuses are only INACTIVE and ACTIVE; everything operational below is a sub-status").
Sub-status	A clone of a main status with operational semantics. From ACTIVE: active, complementary discount, DND active, seasonal disconnect, VIP, staff account. From INACTIVE/SUSPENDED: inactive pending customer, inactive pending VIP, seasonal disconnect, relocating awaiting construction, hold, forget me, duplicate, no-disturb, disconnect staff, churned, awaiting equipment collection.
Service class	VIP / Platinum / Gold / STAFF tier flag on an Account. Informational only — no billing impact per workshop rule. Drives Support routing (VIP gets internal contractor; see WO module FLOW-INSTALLATION + FLOW-SUPPORT Drools rules).
NPD	"No Payment Done" — historically a boolean column on the Account. V3 represents it as one entry in the Account flag system (§3.5), with <code>flag_code='NPD'</code> evaluated daily by a Drools worker against payment history. ~42,000 customers tagged at workshop time. Used as a risk signal for activities like cash-only enforcement and reduced trust in self-service operations. BIL-04 dunning reads this flag when deciding escalation.
Account flag system	Operator-extensible registry of per-Account flags (NPD, churn risk, fraud suspected, high-value, loyalty tier, etc.). See §3.5. Each flag is defined in <code>customer_account_flag_catalog</code> per operator with a value kind (BOOLEAN, SCORE_0_100, TIER, COUNT) and evaluator (MANUAL, DROOLS, EVENT_DRIVEN). Per-account current state lives in <code>customer_account_flag</code> . A daily Drools worker (<code>flag-evaluator-<operator>-v<n></code>) re-evaluates flags marked with <code>evaluator='DROOLS'</code> and writes transitions.
KYC	Know Your Customer — the identity verification flow before service dispatch. Two-level approval: L1 (regional / franchise supervisor) → Team Leader (Wananchi). Outcomes: APPROVED / REJECTED. Stored on Customer; affects the dummy-account-to-real-account conversion at activation time.

Term	Meaning
Attention banner	A yellow flag at the top of an account view in BO UI displaying a high-priority operational note ("Day Zero", "call back", "VIP — handle carefully", "NPD — cash-only enforcement"). Per-account (NOT per-customer) — a customer with multiple accounts may have a banner on one and none on the others. Held on <code>customer_account.attention_banner</code> .
Last Note	The most recent free-text note on the customer record, timestamped with author.
Structured Interaction History	A categorized log of customer interactions per CSR action: Agent name, Call reason, Findings, Resolution (ticket #), VOC (Voice of Customer verbatim). Separate from free-text notes.

2. Customer master — the CAS hierarchy

2.1 Customer entity

The root of the hierarchy. One row per legal entity (person or business).

```

CREATE TABLE customer (
  customer_id          TEXT PRIMARY KEY,          -- 'cust_01HX...' ULID
  operator_code        TEXT NOT NULL,             -- 'WIK' | 'WUG' | 'WTZ' | 'YASSN'
  type                 TEXT NOT NULL,             -- 'RES' | 'COM'
  name                 TEXT NOT NULL,             -- 'Jane Mwangi' | 'Acme Logistics Ltd'
  identification_type_1 TEXT NULL,                 -- 'NATIONAL_ID' | 'PASSPORT' | 'BUSINESS_REG' | 'ECOWAS_CARD' | 'VOT
  identification_number_1 TEXT NULL,              -- per identification_type_1
  identification_type_2 TEXT NULL,                 -- second ID (e.g., ECOWAS card holds both a card-ID and a national-I
  identification_number_2 TEXT NULL,              -- per identification_type_2
  date_of_birth        DATE NULL,                 -- for RES
  business_reg_date    DATE NULL,                 -- for COM
  primary_msisdn       TEXT NOT NULL,             -- '+254712345678' (workshop: required, enforced)
  email                TEXT NULL,                 -- 'jane.mwangi@example.com'
  preferred_language   TEXT NOT NULL DEFAULT 'en', -- per-operator default
  kyc_status           TEXT NOT NULL,             -- 'PENDING' | 'L1_APPROVED' | 'L2_APPROVED' | ... | 'APPROVED' | 'REJE
  -- Derived from customer_kyc_approval rows; maintained by trigger (see
  created_at           TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at           TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_customer_operator ON customer (operator_code);
CREATE INDEX idx_customer_msisdn  ON customer (primary_msisdn);
CREATE INDEX idx_customer_email   ON customer (email) WHERE email IS NOT NULL;
CREATE INDEX idx_customer_idnum_1 ON customer (identification_type_1, identification_number_1)
  WHERE identification_number_1 IS NOT NULL;
CREATE INDEX idx_customer_idnum_2 ON customer (identification_type_2, identification_number_2)
  WHERE identification_number_2 IS NOT NULL;
CREATE INDEX idx_customer_name    ON customer USING GIN (name gin_trgm_ops);

```

The customer entity holds only legal-identity attributes: who Wananchi recognizes (name, type, IDs, KYC, date of birth, contact channels, language preference). Operational attributes — status, service class, attention banner, flags, notification preferences, work orders, invoices, payments — live on `customer_account` (§2.2). This separation is foundational: see §1 "What this module is" for the architectural principle.

Sample data:

customer_id	operator_code	type	name	id_type_1 / id_number_1	id_type_2 / id_number_2	primary_msisdn	kyc_status
cust_01HX3K9M 2P5Q8R1T4W7Y0 Z3N6	WIK	RES	Jane Mwangi	NATIONAL_ID / 28471929	NULL	+254712345678	APPROVED
cust_01HX5M2K 9P3Q7R4T6W8Y0 Z2N4Q	WIK	COM	Karen Hardy Apartments Ltd	BUSINESS_REG / BRS/2019/1872 34	NULL	+254733654321	APPROVED
cust_01HZA1B2 C3D4E5F6G7H8J 9K0L1	WIK	RES	Mike Otieno	NATIONAL_ID / 30912834	NULL	+254728889999	PENDING
cust_01HZP9Q8 R7S6T5U4V3W2X 1Y0Z9	WIK	RES	Hon. Sarah Wanjiku	NATIONAL_ID / 20543219	NULL	+254710001122	APPROVED
cust_01HZ_PET ER_KIMANI_NPD	WIK	RES	Peter Kimani	NATIONAL_ID / 15672345	NULL	+254799887766	APPROVED

customer_id	operator_code	type	name	id_type_1 / id_number_1	id_type_2 / id_number_2	primary_msisdn	kyc_status
cust_01HZ_YAS SN_ECOWAS_EXA MPLE	YASSN	RES	Aissatou Diop	ECOWAS_CARD / CI-SN-1987345 22	NATIONAL_ID_S N / 1198195400123	+221771234567	APPROVED

Kenya customers (rows 1–5) use _1 only with _2 NULL — KE workshop pattern is one identification at a time (alternatives: National ID, Passport, Business Registration). The YASSN row demonstrates the ECOWAS pattern where both the ECOWAS card-ID and the national ID are captured on the same customer; both are first-class indexed for search.

2.2 Account entity

One row per deployment (one customer's physical service location). Workshop rule: "1 account = 1 deployment."
Multi-location customers in Kenya hold multiple Account rows.

```

CREATE TABLE customer_account (
  account_id          TEXT PRIMARY KEY,          -- 'acct_01HX...' ULID
  account_number      TEXT NOT NULL UNIQUE,      -- '002-0598570T' workshop format; THE operational key used by all
  payment_account_number TEXT NULL,              -- the reference the customer uses when paying (M-Pesa Paybill account)
  customer_id         TEXT NOT NULL REFERENCES customer(customer_id),
  operator_code       TEXT NOT NULL,             -- 'WIK' | etc
  homepass_id        TEXT NULL,                  -- FK to RLM-CFG-01 (the physical address)
  service_address     TEXT NOT NULL,             -- denormalized for display, free-text per workshop
  status             TEXT NOT NULL,              -- 'INACTIVE' | 'ACTIVE'
  sub_status         TEXT NOT NULL,              -- see §3 sub-status registry
  sub_status_reason   TEXT NULL,                 -- free text, e.g. 'dunning escalation L3'
  sub_status_changed_at TIMESTAMP NOT NULL DEFAULT NOW(),
  service_class_1     TEXT NULL,                 -- primary tier flag – 'VIP' | 'PLATINUM' | 'GOLD' | 'STAFF' | operator-defined
  service_class_2     TEXT NULL,                 -- orthogonal secondary classification – operator-defined (e.g., region)
  service_class_3     TEXT NULL,                 -- orthogonal tertiary classification – operator-defined (e.g., region)
  attention_banner    TEXT NULL,                 -- free text; surfaced at top of account view in BO UI. Single banner
  subscription_id     TEXT NULL,                 -- FK to SUB-LM-01 subscription; 1:1 – at most one non-terminated subscription
  start_bill_date     DATE NULL,                 -- workshop's 'Start Bill'
  install_date        DATE NULL,                 -- workshop's 'Install Date'
  disconnect_date     DATE NULL,                 -- workshop's 'Disconnect'
  account_manager_id  TEXT NULL,                 -- BO user id (commercial owner)
  franchise_code      TEXT NULL,                 -- which franchise registered this account
  created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE UNIQUE INDEX uq_account_customer_homepass
  ON customer_account (customer_id, homepass_id)
  WHERE homepass_id IS NOT NULL AND status != 'CHURNED_RECYCLED'; -- per workshop: 1 customer × 1 deployment = 1 account

CREATE UNIQUE INDEX uq_account_subscription
  ON customer_account (subscription_id)
  WHERE subscription_id IS NOT NULL; -- 1:1 – a subscription belongs to exactly one account

CREATE INDEX idx_account_customer      ON customer_account (customer_id);
CREATE INDEX idx_account_homepass     ON customer_account (homepass_id);
CREATE INDEX idx_account_subscription ON customer_account (subscription_id);
CREATE INDEX idx_account_status       ON customer_account (status, sub_status);
CREATE INDEX idx_account_operator     ON customer_account (operator_code);
CREATE INDEX idx_account_franchise    ON customer_account (franchise_code);
CREATE UNIQUE INDEX uq_account_number ON customer_account (operator_code, account_number);
CREATE UNIQUE INDEX uq_payment_number ON customer_account (operator_code, payment_account_number)
  WHERE payment_account_number IS NOT NULL; -- per-operator uniqueness for gateway-callback resolution

```

The `subscription_id` column carries the 1:1 link from this Account to its Subscription (owned by SUB-LM-01). Populated when the order completes and the subscription record is created (typically by FUL-02 STEP-SUBSCRIPTION). NULL between account creation and order completion. The `uq_account_subscription` partial unique index enforces the 1:1 rule at the DB layer: no two accounts may share the same `subscription_id`.

The combined uniqueness constraints `uq_account_customer_homepass` + `uq_account_subscription` enforce the workshop's "1 account = 1 deployment" rule alongside "1 account ↔ 1 subscription".

Two account numbers — internal and payment. The Account carries two distinct identifiers:

- `account_number` — SOPHIX-generated at account creation. The operational key used by every internal team (CSR, BO dispatcher, finance, technician). Workshop's "the only operational key across all sections and teams." Format 002-0598570K. Immutable.
- `payment_account_number` — the reference the customer uses when making a payment. This is what the payment gateway (M-Pesa Paybill, bank transfer reference, USSD reference) returns in its callback, and what BIL-01 / BIL-05 use

to resolve "this payment of KES 2,500 with reference 598570 belongs to which account." May be auto-populated from `account_number` at creation (operator-config decides), or supplied independently. Mutable — can be updated if the customer's payment reference changes (e.g., operator migrates to a new gateway with a different reference format). NULL between account creation and the moment a payment reference is assigned. Uniqueness is enforced per-operator via the `uq_payment_account_number` partial index — gateway callbacks always resolve to exactly one account within an operator's tenant.

Sample data:

account_id	account_number	payment_account_number	service_address	status	sub_status	service_class_1	attention_banner
acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7	002-0598570K	598570	Karen Hardy, Karen, Nairobi	ACTIVE	active	NULL	NULL
acct_01HX5M2K9P3Q7R4T6W8Y0Z2N5R	002-0612847C	612847	Karen Hardy Apartments, Block C, Karen	ACTIVE	active	PLATINUM	NULL
acct_01HZA1B2C3D4E5F6G7H8J9K0L2	002-0701432P	701432	Eastlands, Doonholm Road 28, Apt 7B	INACTIVE	inactive pending customer	NULL	NULL
acct_01HZP9Q8R7S6T5U4V3W2X1Y0Za	002-0432156V	432156	Lavington, Othaya Road 14, House 3A	ACTIVE	VIP	VIP	VIP — county MP. CSRs must escalate first.
acct_01HZ_PETER_KIMANI_NPD_ACCT	002-0712986N	712986	Kibera, Olympic Estate, Block 4	ACTIVE	active	NULL	NPD — cash-only enforcement.
acct_01HZ_SEASONAL_EXAMPLE_0001	002-0589321S	589321	Diani Beach, Cottage 47	INACTIVE	seasonal disconnect	NULL	NULL
acct_01HZ_YASN_EXAMPLE_0001	SN-002-0173045X	WSN173045	Dakar, Plateau, Avenue Pompidou	ACTIVE	active	NULL	NULL

Customer `cust_01HZA1...L1` (Mike Otieno) holds TWO accounts (rows 3 and 6) — one inactive pending customer at Eastlands, one seasonal disconnect at Diani. This is the multi-location customer pattern in Kenya — separate accounts, separate sub-statuses, separate service classes, separate attention banners, all linked to the same customer.

Note: row 4 (Hon. Sarah Wanjiku's account) carries both `service_class_1` = VIP AND `attention_banner` — service class drives Drools-based routing rules in WO module; attention banner is the freeform CSR-visible alert. They are independent.

Row 5 (Peter Kimani's account) demonstrates the post-v1.6 model: NPD lives in the flag system (§3.5) as `customer_account_flag` rows; the account-level `attention_banner` is the CSR-facing text reminder that Drools-evaluated flags are surfaced operationally.

2.3 Subscription linkage — owned by SUB-LM-01, referenced from Account

ILM-CFG-01 does **not** own a subscriber/subscription entity. The workshop's "Subscriber" tier of the CAS model maps to SUB-LM-01's subscription entity, which is the V3 first-class record of a service instance (one row per package delivered to a customer at a HomePass).

The link is materialized via `customer_account.subscription_id` (added in §2.2). **The relationship is 1:1:** each Account is linked to at most one Subscription; each Subscription belongs to exactly one Account. Multi-service customers (Internet + TV + Voice) hold a single Subscription whose Package contains multiple Services — the multi-service decomposition lives in PLM-CFG-01 + SIP-01, not in additional ILM/SUB-LM rows.

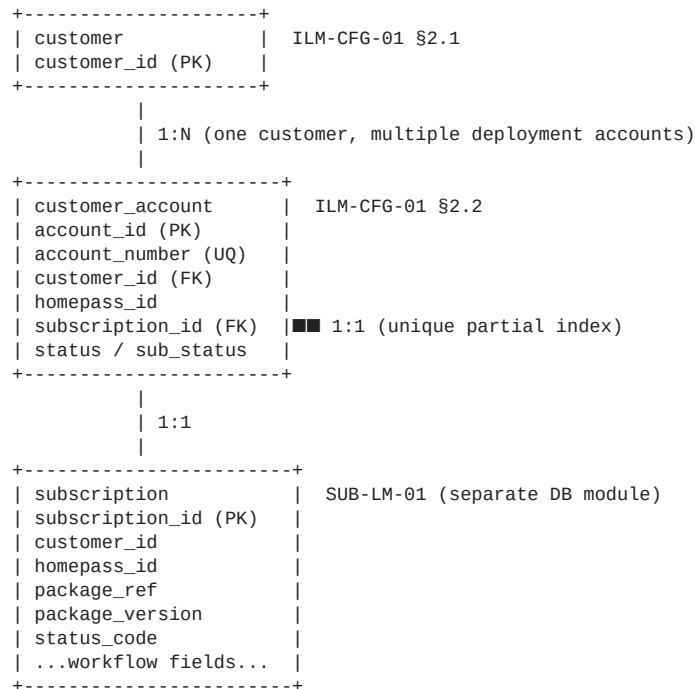
SUB-LM-01's subscription table carries:

- `subscription_id` — PK; the value referenced from `customer_account.subscription_id`
- `customer_id` — links to ILM-CFG-01's customer (must match `customer_account.customer_id`)
- `homepass_id` — links to RLM-CFG-01's HomePass (must match `customer_account.homepass_id`)
- `package_ref` + `package_version` — links to SIP-01's package catalog

- status_code — the subscription lifecycle state (ACTIVE / PAUSED / SUSPENDED_NP / TERMINATED / etc.)
- All the workflow-driven state-transition data

Multi-location customers: each Account holds its own subscription_id pointing to a separate Subscription record. Customer Otieno with accounts at Eastlands and Diani has two Subscriptions, one per Account.

2.4 The 1:1 Account ↔ Subscription model



The 1:1 Account ↔ Subscription rule. Every Account is linked to exactly one Subscription; every Subscription belongs to exactly one Account. The Account's lifecycle (ACTIVE / INACTIVE + sub-statuses) mirrors its single Subscription's lifecycle — there is no ambiguity from one Subscription being ACTIVE while another on the same Account is SUSPENDED, because no Account ever holds more than one Subscription.

Multi-service customers are modelled at the Package layer, not by adding Subscriptions. A customer with Internet + TV + VOIP at one HomePass holds ONE Subscription whose Package contains three Services. The multi-Service decomposition lives in PLM-CFG-01 + SIP-01 — the Package catalog defines which Services compose a given Package. Service-level state (e.g., TV temporarily disabled while Internet stays on) is owned by PLM-CFG-01 / SUB-LM-01 and does not change the Account's sub-status. The Account stays ACTIVE as long as the Subscription has at least one non-terminated Service.

Multi-Subscription customers — a customer with two service locations holds two Accounts, each at a different HomePass (per uq_account_customer_homepass), each with its own subscription_id pointing to a separate Subscription. The customer-level view (single pane of glass) shows both Accounts and walks to both Subscriptions via the FK.

Workshop "1 account = 1 deployment" rule preserved. Each Account corresponds to exactly one physical deployment (one HomePass) and delivers exactly one Subscription (one Package) there. A customer adding TV to their existing Internet does NOT get a second Account — they get a Package upgrade on their existing Subscription (MACD Change-of-Service workflow in SUB-LM-01).

MACD operations. When a customer Changes their package (upgrade / downgrade): SUB-LM-01's MACD workflow either updates the existing Subscription row in-place (same subscription_id, new Package) or creates a new Subscription and terminates the old one. In the terminate-and-recreate case, the Account's subscription_id is updated to point to the new Subscription via the PUT subscription endpoint; the previous Subscription row stays in SUB-LM-01 with status=TERMINATED for audit but is no longer referenced from any Account. In the in-place case, subscription_id is unchanged.

Account ↔ Invoice relationship. Because each Account is linked 1:1 to one Subscription, each Account also corresponds to exactly one billing stream. BIL-01 (Charging Engine) generates charges keyed by subscriptionId; BIL-02-GEN-01 (Invoice Generation) attaches those charges as line items to the Invoice for the corresponding Account's billing cycle. So per billing cycle, per Account, there is **exactly one Invoice** — containing the line items for the Services delivered by that Account's single Subscription's Package.

Concretely:

- A customer with multi-service at one location (Internet + TV + VOIP in one Package on one Account) gets **one Invoice per cycle** with multiple line items (one per Service in the Package).
- A customer with two service locations (two Accounts, two Subscriptions) gets **two Invoices per cycle** — one per Account, each containing the line items for that Account's Subscription.
- Dunning operates per-Account (BIL-04): an Account whose Invoice goes unpaid is dunned independently of the customer's other Accounts.
- Payment reconciliation uses `customer_account.payment_account_number` to resolve the inbound gateway callback to the Account, then applies the payment against that Account's open Invoice(s).

2.5 The Customer/Account/Subscription view in one query

"Show me everything for Jane Mwangi" — a CSR's view as they open her record:

ILM-CFG-01 returns customer + accounts:

```
SELECT
  c.customer_id, c.name, c.primary_msisdn, c.email,
  a.account_id, a.account_number, a.service_address,
  a.status, a.sub_status, a.service_class_1, a.service_class_2, a.service_class_3,
  a.attention_banner, a.homepass_id, a.subscription_id,
  a.install_date, a.account_manager_id, a.franchise_code
FROM customer c
LEFT JOIN customer_account a ON a.customer_id = c.customer_id
WHERE c.customer_id = 'cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6';
```

The CSR's UI then makes a single direct call to SUB-LM-01 per account using the `subscription_id` from the row:

```
GET /api/subscriptions/subs_01HX_JANE_TRIPLE_PLAY
```

Returns the customer's Subscription record with all package + workflow + service-derivation data. No need to query by (`customer_id`, `homepass_id`) — the direct FK is the primary lookup path.

For Jane: 1 customer × 1 account × 1 subscription. For the multi-location Otieno: 1 customer × 2 accounts × 2 subscriptions (looked up by FK from each account).

2.6 Reconciling the workshop's CAS naming

Workshop term	V3 entity	Where owned	Link
Customer	customer	ILM-CFG-01 (this DD)	—
Account	customer_account	ILM-CFG-01 (this DD)	customer_id FK
Subscriber	subscription	SUB-LM-01 (separate DD)	customer_account.subscription_id FK (1:1)

ILM owns C and A; SUB-LM-01 owns S. The 1:1 link from Account to Subscription via `subscription_id` materializes the workshop's CAS hierarchy without ILM duplicating SUB-LM-01's data. Workshop calls the tier "Subscriber"; V3 calls the entity "Subscription"; the role is the same.

3. Status registry — main statuses + sub-statuses

3.1 Sub-status catalog

The sub-status enum is operator-configurable. v1.0 seeds KE WIK from the workshop sub-status list.

```
CREATE TABLE customer_sub_status_catalog (
  operator_code      TEXT NOT NULL,
  sub_status_code    TEXT NOT NULL,
  display_name       TEXT NOT NULL,
  cloned_from        TEXT NOT NULL,          -- 'ACTIVE' | 'INACTIVE'
  requires_approval  BOOLEAN NOT NULL DEFAULT FALSE,
  approval_roles_jsonb JSONB NULL,          -- array of role codes that can approve
  affects_provisioning BOOLEAN NOT NULL DEFAULT FALSE, -- if TRUE, transition emits to FUL-03/provisioning
  customer_visible    BOOLEAN NOT NULL DEFAULT TRUE,
  PRIMARY KEY (operator_code, sub_status_code)
);
```

Sample v1.0 WIK seed:

operator_code	sub_status_code	display_name	cloned_from	requires_approval	affects_provisioning	customer_visible
WIK	active	Active	ACTIVE	false	true	true
WIK	complementary discount	Complimentary Discount	ACTIVE	true	false	true
WIK	DND active	Do Not Disturb – Active	ACTIVE	false	false	false
WIK	seasonal disconnect	Seasonal Disconnect (travel)	ACTIVE	false	true	true
WIK	VIP	VIP	ACTIVE	true	false	false
WIK	staff account	Staff Account	ACTIVE	true	false	false
WIK	inactive pending customer	Inactive Pending Customer	INACTIVE	false	true	true
WIK	inactive pending VIP	Inactive Pending VIP	INACTIVE	true	true	false
WIK	seasonal disconnect (inactive)	Seasonal Disconnect – Inactive	INACTIVE	false	true	true
WIK	relocating awaiting construction	Relocating – Awaiting Construction	INACTIVE	false	false	true
WIK	hold	Hold	INACTIVE	true	true	false
WIK	forget me	Forget Me (GDPR)	INACTIVE	true	true	false
WIK	duplicate	Duplicate	INACTIVE	true	false	false
WIK	no-disturb	No Disturb	INACTIVE	false	false	false
WIK	disconnect staff	Staff Disconnect	INACTIVE	true	true	false
WIK	churned	Churned	INACTIVE	true	true	true
WIK	awaiting equipment collection	Awaiting Equipment Collection	INACTIVE	false	false	true

3.2 Status change history

Every transition is audited.

```
CREATE TABLE customer_account_status_history (
  history_id          BIGSERIAL PRIMARY KEY,
  account_id         TEXT NOT NULL REFERENCES customer_account(account_id),
  customer_id        TEXT NOT NULL,                -- denormalized for fast lookup
  prev_status        TEXT NOT NULL,
  prev_sub_status    TEXT NOT NULL,
  new_status         TEXT NOT NULL,
  new_sub_status     TEXT NOT NULL,
  change_reason      TEXT NULL,
  changed_by         TEXT NOT NULL,                -- B0 user id or 'system:bil-04' for dunning
  changed_by_role    TEXT NOT NULL,                -- 'CSR' | 'BO_AGENT' | 'BILLING_TEAM' | 'SYSTEM'
  approval_reference TEXT NULL,                    -- external approval ticket ref when applicable
  request_channel    TEXT NULL,                    -- 'EMAIL' | 'CALL_CENTER' | 'CUSTOMER_APP' | 'ONBOARDING' | 'SYSTEM'
  source_event_ref   TEXT NULL,                    -- the originating event (e.g., dunning event id)
  changed_at         TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_status_hist_account ON customer_account_status_history (account_id, changed_at);
CREATE INDEX idx_status_hist_customer ON customer_account_status_history (customer_id, changed_at);
```

Sample rows:

history_id	account_id	prev_status	prev_sub_status	new_status	new_sub_status	change_reason	changed_by	changed_by_role	changed_at
1	acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7	INACTIVE	inactive pending customer	ACTIVE	active	Activation after install	system:full-02	SYSTEM	2026-02-17 14:30:00

history_id	account_id	prev_status	prev_sub_status	new_status	new_sub_status	change_reason	changed_by	changed_by_role	changed_at
2	acct_01HZA1B2C3D4E5F6G7H8J9K0L2	ACTIVE	active	INACTIVE	inactive pending customer	Dunning escalation L3	system:bil-04	SYSTEM	2026-04-22 09:15:00
3	acct_01HZ_SEASONAL_EXAMPLE_0001	ACTIVE	active	INACTIVE	seasonal disconnect	Customer email request: travel until October	bo_user_42	BO_AGENT	2025-07-10 10:00:00
4	acct_01HZP9Q8R7S6T5U4V3W2X1Y0Za	ACTIVE	active	ACTIVE	VIP	Service-class tier escalation; approved by wananchi exec	bo_user_supervisor_03	BO_AGENT	2025-09-04 14:22:00

3.3 Status-change rules

The framework enforces a small set of universal rules across all operator status changes:

- **R-ILM-S-1.** Every `customer_account.status` or `sub_status` change writes a row to `customer_account_status_history`. Enforced via DB trigger on update. The history table is append-only.
- **R-ILM-S-2.** When `customer_sub_status_catalog.requires_approval = TRUE`, the change must include a non-null `approval_reference`. Enforced at the API layer.
- **R-ILM-S-3.** When `customer_sub_status_catalog.affects_provisioning = TRUE`, the change emits `CustomerAccountStatusChanged` event (§7) carrying the transition; FUL-03 / provisioning consumes this to apply network-level changes.
- **R-ILM-S-4.** Service-class tier changes are recorded on `customer_account.service_class` and also generate a `customer_account_status_history` entry where `prev_sub_status = new_sub_status` (the sub-status didn't change; only the tier flag did). Per workshop rule: service class is informational only; no billing impact, no provisioning impact.
- **R-ILM-S-5.** A status change initiated by another module (BIL-04 dunning, WO module Shifting flow, FUL-02 STEP-ACTIVATION) carries `source_event_ref` pointing to the originating event ID for traceability.
- **R-ILM-S-6.** When `sub_status` changes on an Account, that Account's `attention_banner` may also be auto-set per operator-config (e.g., entering `inactive pending customer` could auto-set "Account in dunning" on THAT account only — not on the customer's other accounts); v1.0 ships with this rule disabled — manual banner updates only. Operator-config can enable per sub-status.

3.4 Status transitions triggered externally

Other modules update customer account status by emitting Kafka events that ILM-CFG-01 consumes. The owning module of the status change is recorded in `changed_by`.

Originating module	Event consumed	Status side-effect
FUL-02 STEP-ACTIVATION	OrderActivated	account → ACTIVE active. Subscription status changes are owned by SUB-LM-01's activation workflow, not by ILM.
WO module FLOW-SHIFTING (DISCONNECT phase)	WorkOrderPhaseFinalized with <code>accountSubStatusChange = {to: INA}</code>	account → INACTIVE inactive pending customer (or operator-defined target)
WO module FLOW-SHIFTING (RECONNECT phase)	WorkOrderPhaseFinalized with <code>accountSubStatusChange = {to: STF}</code>	account → ACTIVE staff account (or operator-defined target; the STF workshop sub-status code maps to a specific catalog row)
BIL-04 dunning	DunningLevelEntered (level 3 = SUSPENDED)	account → INACTIVE inactive pending customer
BIL-04 dunning	DunningRecovered	account → ACTIVE active

Originating module	Event consumed	Status side-effect
SUB-LM-01 subscription pause	SubscriptionPaused	NO account-level side-effect by default. Per-operator config can choose to derive an account-level sub-status from subscription state (e.g., "all subscriptions paused → account sub_status = seasonal disconnect"); v1.0 KE ships with this derivation OFF.
SUB-LM-01 subscription resume	SubscriptionResumed	NO account-level side-effect by default.
Manual BO (CSR, BO Agent)	direct PUT /customer-accounts/{id}/sub-status	per the call's payload

The framework's R-ILM-S-5 ensures source attribution. Consumers in ILM-CFG-01 are idempotent on (source_event_ref) so duplicate events don't double-write history.

3.5 Account flag system

Separate from status / sub-status, every Account carries a set of operational **flags** — boolean or scored signals used for risk management, segmentation, dunning policy, fraud detection. Workshop's NPD is one such flag; the system is extensible to many more (churn risk, fraud suspected, high-value, loyalty tier, payment-delay frequency, etc.).

Three components:

1. **Operator-scoped catalog** of flag types — defines what flags exist for an operator, how each is evaluated, what values it can hold.
2. **Per-account flag state** — one row per (account_id, flag_code), holding current value + evaluator + timestamp.
3. **Daily Drools worker** — re-evaluates every Account against the catalog's Drools rules; writes/updates customer_account_flag rows; emits Kafka events for any flag transitions.

3.5.1 Catalog: customer_account_flag_catalog

```
CREATE TABLE customer_account_flag_catalog (
  operator_code      TEXT NOT NULL,
  flag_code          TEXT NOT NULL,
  display_name       TEXT NOT NULL,
  description        TEXT NULL,
  value_kind         TEXT NOT NULL,      -- 'BOOLEAN' | 'SCORE_0_100' | 'TIER' | 'COUNT'
  evaluator          TEXT NOT NULL,      -- 'MANUAL' | 'DROOLS' | 'EVENT_DRIVEN'
  drools_rule_id     TEXT NULL,          -- KJAR rule identifier when evaluator='DROOLS'
  refresh_schedule_cron TEXT NULL,      -- e.g., '0 2 * * *' = daily 02:00; NULL for manual/event-driven
  affects_dunning    BOOLEAN NOT NULL DEFAULT FALSE,
  affects_provisioning BOOLEAN NOT NULL DEFAULT FALSE,
  customer_visible   BOOLEAN NOT NULL DEFAULT FALSE,
  PRIMARY KEY (operator_code, flag_code)
);
```

Sample v1.0 WIK seed:

operator_code	flag_code	display_name	value_kind	evaluator	drools_rule_id	refresh_schedule_cron	affects_dunning
WIK	NPD	No Payment Done	BOOLEAN	DROOLS	flag-evaluator-wik-v1::npd-rule	0 2 * * *	true
WIK	CHURN_RISK	Churn Risk Score	SCORE_0_100	DROOLS	flag-evaluator-wik-v1::churn-risk-score	0 3 * * *	false
WIK	FRAUD_SUSPECTED	Fraud Suspected	BOOLEAN	MANUAL	NULL	NULL	false
WIK	PAYMENT_DELAY_FREQUENT	Frequent Payment Delays	BOOLEAN	DROOLS	flag-evaluator-wik-v1::payment-delay-frequent	0 2 * * *	true
WIK	LOYALTY_TIER	Loyalty Tier	TIER	DROOLS	flag-evaluator-wik-v1::loyalty-tier	0 4 1 * *	false
WIK	HIGH_VALUE	High Value Customer	BOOLEAN	DROOLS	flag-evaluator-wik-v1::high-value	0 4 1 * *	false

YASSN may seed a different set (e.g., add MOBILE_MONEY_PREFERRED flag, swap NPD logic). Operator-scoped catalog supports the regional flexibility principle (memory rule #18).

3.5.2 Per-account state: customer_account_flag

```
CREATE TABLE customer_account_flag (
  account_id          TEXT NOT NULL REFERENCES customer_account(account_id),
  flag_code           TEXT NOT NULL,
  operator_code       TEXT NOT NULL,
  value_text          TEXT NULL,           -- string representation; type per catalog.value_kind
  value_numeric       NUMERIC NULL,       -- for SCORE_0_100, COUNT
  value_boolean       BOOLEAN NULL,       -- for BOOLEAN
  evaluated_by        TEXT NOT NULL,      -- 'system:flag-evaluator-wik-v1' | BO user id | 'system:event:C
  evaluator_kind      TEXT NOT NULL,      -- 'MANUAL' | 'DROOLS' | 'EVENT_DRIVEN'
  evaluation_context  JSONB NULL,         -- snapshot of inputs used by Drools (for audit)
  evaluated_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  expires_at          TIMESTAMP NULL,     -- optional TTL; NULL = no expiry
  PRIMARY KEY (account_id, flag_code),
  FOREIGN KEY (operator_code, flag_code)
    REFERENCES customer_account_flag_catalog(operator_code, flag_code)
);

CREATE INDEX idx_flag_account      ON customer_account_flag (account_id);
CREATE INDEX idx_flag_code        ON customer_account_flag (operator_code, flag_code);
CREATE INDEX idx_flag_active      ON customer_account_flag (operator_code, flag_code, value_boolean)
  WHERE value_boolean = TRUE;
CREATE INDEX idx_flag_evaluated   ON customer_account_flag (evaluated_at);
```

Sample data:

account_id	flag_code	value_boolean	value_numeric	evaluated_by	evaluator_kind	evaluated_at
acct_01HZ_PETER_KIMANI_NPD_ACCOUNT	NPD	true	NULL	system:flag-evaluator-wik-v1	DROOLS	2026-05-20 02:00:14
acct_01HZ_PETER_KIMANI_NPD_ACCOUNT	PAYMENT_DELAY_FREQUENT	true	NULL	system:flag-evaluator-wik-v1	DROOLS	2026-05-20 02:00:14
acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7	CHURN_RISK	NULL	12	system:flag-evaluator-wik-v1	DROOLS	2026-05-20 03:00:22
acct_01HX5M2K9P3Q7R4T6W8Y0Z2N5R	HIGH_VALUE	true	NULL	system:flag-evaluator-wik-v1	DROOLS	2026-05-01 04:00:09
acct_01HZP9Q8R7S6T5U4V3W2X1Y0Za	FRAUD_SUSPECTED	true	NULL	bo_user_fraud_team_02	MANUAL	2026-04-12 11:30:00

3.5.3 Daily Drools evaluator worker

A Camunda external task worker — `flag-evaluator-<operator>-v<n>` — runs on the operator's configured schedule (typically nightly per `customer_account_flag_catalog.refresh_schedule_cron`). For each Account in the operator's tenant:

1. Load the account's recent context: last N payments (from BIL-05), open invoices (from BIL-02-GEN-01), current sub_status, subscription state, age of account.
2. For each `flag_code` in the catalog with `evaluator='DROOLS'`: invoke the corresponding KJAR rule with the context.
3. Compare the rule output to the existing `customer_account_flag.value_*`; if different, write a new value + emit `CustomerAccountFlagChanged` Kafka event.
4. Update `evaluated_at` even when value is unchanged (so consumers can see "last checked at" without inferring from history).

The worker is operator-scoped — `flag-evaluator-wik-v1` is one Camunda job, `flag-evaluator-yassn-v1` is another. Each operator's Drools KJAR can be deployed and versioned independently.

```
POST /engine-rest/external-task/fetchAndLock HTTP/1.1
{
  "workerId": "flag-evaluator-wik-v1-instance-3",
  "maxTasks": 100,
  "topics": [{"topicName": "sophix.ilm-cfg-01.flag-evaluation", "lockDuration": 300000}]
}
```

Each fetched task carries an account batch (typically 1,000 accounts) — the worker processes the batch, completes the task, fetches the next. Long jobs split across multiple Camunda task instances.

3.5.4 Flag rules

Rule	Description
R-ILM-F-1	Flag value_* columns are typed per the catalog's value_kind: BOOLEAN → value_boolean; SCORE_0_100/COUNT → value_numeric; TIER → value_text. Workers MUST populate the column matching the kind; others stay NULL.
R-ILM-F-2	A flag transition (value change, set, or clear) emits CustomerAccountFlagChanged event (§8). The event carries old + new values + evaluator + evaluation context.
R-ILM-F-3	Flags with catalog.affects_dunning=TRUE are read by BIL-04 dunning before deciding dunning level (e.g., NPD=true may trigger faster escalation).
R-ILM-F-4	Flags with catalog.affects_provisioning=TRUE are read by FUL-03 / provisioning during state changes (e.g., FRAUD_SUSPECTED=true blocks new service activation).
R-ILM-F-5	Flags with catalog.customer_visible=TRUE are surfaced in customer-facing UIs (e.g., loyalty tier). Customer-invisible flags (NPD, fraud, churn risk) are BO-only.
R-ILM-F-6	Flags with evaluator='MANUAL' can only be set via the dedicated API endpoint (PUT /api/customer-accounts/{id}/flags/{flag_code}) by a BO user. Workers MUST NOT write manual flags.
R-ILM-F-7	Flag history is captured by Kafka event log (downstream CVM / data warehouse consumes), NOT by an internal customer_account_flag_history table. The current state is sufficient for ILM-CFG-01's operational needs.
R-ILM-F-8	Drools workers MUST capture the evaluation_context snapshot when writing the flag — for audit reproducibility (e.g., "why was NPD set on 2026-05-20").

4. Contacts, notes, preferences, interactions

4.1 Contact methods

The workshop's "Contact Methods" table — multi-channel addressable identities for the customer. Required-primary-MSISDN rule (R-ILM-C-1 below) is enforced.

```
CREATE TABLE customer_contact_method (
  contact_method_id TEXT PRIMARY KEY, -- 'cm_01HX...' ULID
  customer_id TEXT NOT NULL REFERENCES customer(customer_id),
  channel TEXT NOT NULL, -- 'PHONE' | 'MSISDN' | 'EMAIL' | 'ALT_PHONE' | 'INSTALL_CONTACT'
  value TEXT NOT NULL, -- '+254712345678' | 'jane@example.com'
  is_primary BOOLEAN NOT NULL DEFAULT FALSE,
  is_billing BOOLEAN NOT NULL DEFAULT FALSE, -- 'billing email' per workshop
  label TEXT NULL, -- 'Personal' | 'Office' | 'Daughter'
  verified BOOLEAN NOT NULL DEFAULT FALSE,
  verified_at TIMESTAMP NULL,
  added_by TEXT NOT NULL, -- BO user id or 'agent_pwa:onboarding'
  added_at TIMESTAMP NOT NULL DEFAULT NOW(),
  superseded_by_id TEXT NULL, -- soft-deletion: replaced by this newer entry
  superseded_at TIMESTAMP NULL
);

CREATE INDEX idx_contact_customer ON customer_contact_method (customer_id);
CREATE INDEX idx_contact_value ON customer_contact_method (channel, value);
CREATE UNIQUE INDEX uq_contact_primary_msisdn
  ON customer_contact_method (customer_id) WHERE channel = 'MSISDN' AND is_primary = TRUE AND superseded_by_id IS NULL;
```

Sample data for Jane Mwangi:

contact_method_id	customer_id	channel	value	is_primary	is_billing	label	verified	added_by
cm_01HX_JAN_E_MSISDN	cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6	MSISDN	+254712345678	true	false	Personal	true	agent_pwa:onboarding
cm_01HX_JAN_E_EMAIL	cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6	EMAIL	jane.mwangi@example.com	true	true	Personal	false	agent_pwa:onboarding
cm_01HX_JAN_E_ALT_PHONE	cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6	ALT_PHONE	+254733998877	false	false	Husband	false	bo_user_csr_07
cm_01HX_JAN_E_INSTALL	cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6	INSTALL_CONTACT	+254700112233	false	false	Building caretaker – Mr. Kamau	false	agent_pwa:onboarding

4.2 Customer notes

Workshop: "Attention banner (yellow flag, e.g. Day Zero, call back) + structured Last Note." V3 holds the Attention banner on `customer_account.attention_banner` — per-account, not per-customer (since the banner is an operational signal about a specific account, e.g., "NPD — cash-only enforcement" applies to one account; a customer with multiple accounts may have different banners on each). Notes form a time-ordered log; notes with `account_id` set drive the matching account's banner (per R-ILM-C-2); notes without `account_id` are customer-level (e.g., "deceased") and don't drive any banner.

```
CREATE TABLE customer_note (
  note_id          TEXT PRIMARY KEY,          -- 'note_01HX...' ULID
  customer_id      TEXT NOT NULL REFERENCES customer(customer_id),
  account_id       TEXT NULL,                 -- optional scope: note about a specific account
  note_kind        TEXT NOT NULL,             -- 'free_text' | 'attention_request' | 'csr_call_log' | 'billing...'
  content_text     TEXT NULL,                 -- free-text body
  content_jsonb    JSONB NULL,                -- structured body when note_kind has schema
  is_attention     BOOLEAN NOT NULL DEFAULT FALSE, -- if TRUE, was the source of the current attention_banner
  added_by         TEXT NOT NULL,             -- 'CSR' | 'BO_AGENT' | 'TECH' | 'SYSTEM'
  added_at         TIMESTAMP NOT NULL DEFAULT NOW(),
  superseded_by_id TEXT NULL,
  superseded_at    TIMESTAMP NULL
);

CREATE INDEX idx_note_customer ON customer_note (customer_id, added_at DESC);
CREATE INDEX idx_note_attention ON customer_note (customer_id, is_attention) WHERE is_attention = TRUE;
```

Sample rows:

note_id	customer_id	note_kind	content (excerpt)	is_attention	added_by	added_by_role	added_at
note_01HX_JAN E_DAY_ZERO	cust_01HX3K9M 2P5Q8R1T4W7Y0 Z3N6	attention_req uest	Day Zero – install tomorrow, customer to call if any issue	true	bo_user_csr_0 7	CSR	2026-02-16 17:00:00
note_01HX_JAN E_AFTER_INSTA LL	cust_01HX3K9M 2P5Q8R1T4W7Y0 Z3N6	csr_call_log	Customer reported speed test 48 Mbps as expected. Service working well.	false	bo_user_csr_1 1	CSR	2026-02-18 10:30:00
note_01HZP_VI P_HANDLING	cust_01HZP9Q8 R7S6T5U4V3W2X 1Y0Z9	attention_req uest	VIP – county MP. CSRs must escalate first.	true	bo_user_super visor_03	BO_AGENT	2025-08-22 09:00:00

4.3 Structured interaction history

Distinct from free-text notes. The workshop's "Structured Last Note entry: Agent name · Call reason · Findings · Resolution (ticket #)VOC (Voice of Customer verbatim)" — a categorized record per customer interaction.

```
CREATE TABLE customer_interaction (
  interaction_id    TEXT PRIMARY KEY,
  customer_id      TEXT NOT NULL REFERENCES customer(customer_id),
  account_id       TEXT NULL,                 -- optional
  channel          TEXT NOT NULL,             -- 'CALL' | 'EMAIL' | 'CUSTOMER_APP' | 'BO_WALK_IN' | 'WHATSAPP'
  call_reason      TEXT NULL,                 -- 'BILLING_QUERY' | 'TECHNICAL_ISSUE' | 'UPGRADE_INTENT' | 'CANC...'
  findings         TEXT NULL,                 -- agent's free-text findings
  resolution_summary TEXT NULL,               -- agent's free-text resolution
  ticket_ref       TEXT NULL,                 -- future Ticketing module ref; NULL in v1.0
  voc_verbatim     TEXT NULL,                 -- customer's exact words
  agent_id         TEXT NOT NULL,             -- CSR or BO agent
  agent_role       TEXT NOT NULL,
  duration_seconds INTEGER NULL,              -- for CALL
  recorded_at      TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_interaction_customer ON customer_interaction (customer_id, recorded_at DESC);
CREATE INDEX idx_interaction_account ON customer_interaction (account_id, recorded_at DESC);
```

Sample row:

interaction_id	customer_id	channel	call_reason	findings	resolution_summary	agent_id	duration_seconds	recorded_at
intx_01HX_JANE_CALL_1	cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6	CALL	TECHNICAL_ISSUE	Customer reports intermittent connectivity Monday morning	Reset session on OLT port. Customer back up. Logged w/o for tech if recurs.	bo_user_csr_11	480	2026-04-15 09:22:00

4.4 Notification preferences (account-scoped)

Notification preferences are scoped to **the Account**, not the Customer. A customer with multiple Accounts (one per service location) may have different notification rules per Account — e.g., the Karen apartment uses email-only with auto-suspension reminders; the Diani holiday cottage is in seasonal disconnect and the customer wants only critical-failure SMS alerts. Customer-level preference would force one set across all Accounts and is therefore wrong.

```
CREATE TABLE customer_account_notification_pref (
  account_id      TEXT PRIMARY KEY REFERENCES customer_account(account_id),
  customer_id     TEXT NOT NULL,                -- denormalized for cross-account queries
  email_optin     BOOLEAN NOT NULL DEFAULT TRUE,
  sms_optin       BOOLEAN NOT NULL DEFAULT TRUE,
  preferred_channel TEXT NOT NULL DEFAULT 'EMAIL', -- 'EMAIL' | 'SMS' | 'BOTH'
  dnd_outbound    BOOLEAN NOT NULL DEFAULT FALSE, -- linked to DND sub-status on this account
  notification_topics_jsonb JSONB NOT NULL DEFAULT '{}', -- per-topic preference: {low_balance: true, auto_suspension: true, billing_reminder: true}
  updated_at      TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_by      TEXT NOT NULL
);

CREATE INDEX idx_notif_pref_customer ON customer_account_notification_pref (customer_id);
```

Sample data:

account_id	customer_id	email_optin	sms_optin	preferred_channel	dnd_outbound	notification_topics_jsonb
acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7	cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6	true	true	EMAIL	false	{"low_balance": true, "auto_suspension": true, "billing_reminder": true}
acct_01HZA1B2C3D4E5F6G7H8J9K0L2	cust_01HZA1B2C3D4E5F6G7H8J9K0L1	true	true	EMAIL	false	{"low_balance": true, "auto_suspension": true, "billing_reminder": true}
acct_01HZ_SEASONAL_EXAMPLE_0001	cust_01HZA1B2C3D4E5F6G7H8J9K0L1	false	true	SMS	false	{"low_balance": false, "auto_suspension": false, "billing_reminder": false, "critical_failure": true}

Workshop rule confirmed: "Channel: Email if available, otherwise SMS." The `preferred_channel` column governs DD_NOT-01's send-to-customer decision; DD_NOT-01 reads this preference at notification dispatch time, scoped by `account_id`.

Per workshop: status-related preferences (complimentary discount, DND, seasonal disconnect, etc.) are implemented as sub-statuses on `customer_account` — see §3. The notification-pref table above covers channel + topic preferences only, not status preferences.

4.4.1 Consent ledger

`customer_account_notification_pref` stores the current preference state. It is not enough for audit because it does not prove when consent changed, who captured it, or which channel/source produced the consent. Use the append-only consent ledger for marketing and optional informational communications.

```
CREATE TABLE customer_consent_ledger (
  consent_id      TEXT PRIMARY KEY,
  customer_id     TEXT NOT NULL REFERENCES customer(customer_id),
  account_id      TEXT NOT NULL REFERENCES customer_account(account_id),
  operator_code   TEXT NOT NULL,
  consent_scope   TEXT NOT NULL,                -- MARKETING_SMS, MARKETING_EMAIL, WHATSAPP_INFO, FIELD_KYC_CONFIRMATION
)
```

```

consent_value          BOOLEAN NOT NULL,    -- true=opt-in/confirmed, false=opt-out/refused
capture_channel        TEXT NOT NULL,      -- BACKOFFICE, SELF_CARE, USSD, SALES_APP, CONTRACTOR_APP, IMPORT
capture_actor_id       TEXT NULL,
capture_source_ref     TEXT NULL,          -- ticket/order/work-order/import row where relevant
evidence_file_ref      TEXT NULL,
captured_at            TIMESTAMP NOT NULL DEFAULT NOW(),
effective_from         TIMESTAMP NOT NULL DEFAULT NOW(),
revoked_at             TIMESTAMP NULL,
notes                 TEXT NULL
);

CREATE INDEX idx_consent_customer ON customer_consent_ledger (customer_id, consent_scope, captured_at DESC);
CREATE INDEX idx_consent_account ON customer_consent_ledger (account_id, consent_scope, captured_at DESC);

```

Sample data:

consent_id	customer_id	account_id	consent_scope	consent_value	capture_channel	Meaning
cons_01HZ_SMS_001	cust_01HX...	acct_01HX...	MARKETING_SMS	true	SALES_APP	Customer opted in to marketing SMS during outdoor sale.
cons_01HZ_EMAIL_002	cust_01HX...	acct_01HX...	MARKETING_EMAIL	false	SELF_CARE	Customer opted out of marketing email in portal.

Rule: every marketing opt-in/opt-out change must update current preferences and append a `customer_consent_ledger` row in the same transaction or command boundary. Transactional notifications such as invoices, payment confirmations, outage notices, and security messages do not require marketing consent, but NOT-01 must still respect legal channel restrictions and DND rules where applicable.

4.5 Contact + note + interaction rules

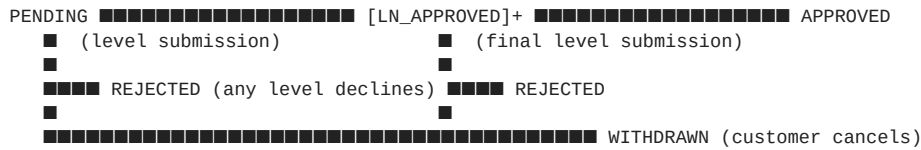
Rule	Description
R-ILM-C-1	Every customer MUST have at least one <code>customer_contact_method</code> row with <code>channel=MSISDN</code> , <code>is_primary=TRUE</code> , <code>superseded_by_id</code> IS NULL. Enforced at: customer creation; primary MSISDN supersession (the replacing row must be inserted in the same transaction). Per workshop: "Every customer must have at least one primary phone (MSISDN) — enforced at capture and at every contact-update operation".
R-ILM-C-2	An Account's <code>attention_banner</code> is auto-derived from the <code>content_text</code> of the most-recent <code>customer_note</code> with <code>is_attention=TRUE</code> , <code>account_id</code> = <this account>, <code>superseded_by_id</code> IS NULL. If no such account-scoped attention note exists, banner is NULL. Notes with <code>account_id</code> IS NULL (customer-level notes) do not drive any account's banner. Updated by trigger.
R-ILM-C-3	Contact methods are NEVER hard-deleted; supersession via <code>superseded_by_id</code> preserves audit trail.
R-ILM-C-4	Notes are NEVER hard-deleted; supersession via <code>superseded_by_id</code> preserves audit trail.
R-ILM-C-5	<code>customer_interaction</code> rows are append-only; mutations are by appending a new interaction.
R-ILM-C-6	DND sub-status on an account (<code>sub_status</code> = 'DND active') automatically sets <code>customer_account_notification_pref.dnd_outbound</code> = TRUE for THAT account only — not for the customer's other accounts. Reverting the sub-status reverts the DND flag on that account. Implemented via Kafka event consumer (eats the <code>CustomerAccountStatusChanged</code> event with <code>sub_status=DND</code> , account-scoped).
R-ILM-C-7	Marketing and optional informational consent changes are append-only in <code>customer_consent_ledger</code> . Current preferences may be updated, but consent evidence must remain immutable.

5. KYC

5.1 Workflow

Per workshop: "Captured by the franchise agent on the Field Agent PWA → L1 validated by the supervisor (regional, franchise) → final approval by a Zuku Team Leader." → "Two-level approval is mandatory before a dummy account is allocated and a payment link is issued."

The customer.kyc_status field is the state machine. v1.0 KE WIK ships with two levels (L1 + Final), but the design supports an arbitrary number of levels per operator (future deployments may insert L2, L3, etc. between L1 and Final without schema or endpoint changes — only catalog config). Transitions:



L1 → APPROVED gate authorizes FUL-02 STEP-VALIDATE's CUSTOMER_ELIGIBLE check.

5.2 KYC approval chain — append-only history

Approvals are recorded as a list (one row per approval action), not as fixed-position columns. This supports any number of levels without schema changes when a future operator deploys more or fewer levels.

```

CREATE TABLE customer_kyc_approval (
  approval_id      TEXT PRIMARY KEY,          -- 'kycappr_01HX...' ULID
  customer_id      TEXT NOT NULL REFERENCES customer(customer_id),
  approval_level   INTEGER NOT NULL,          -- 1, 2, 3, ... (1 = L1)
  approval_level_name TEXT NOT NULL,          -- 'L1_SUPERVISOR' | 'TEAM_LEADER' | operator-defined
  decision         TEXT NOT NULL,             -- 'APPROVED' | 'REJECTED'
  is_final         BOOLEAN NOT NULL,          -- TRUE on the level that closes the chain
  approver_id      TEXT NOT NULL,             -- BO user id
  approver_role    TEXT NOT NULL,             -- FOUNDATION_AUTH role at approval time
  comments         TEXT NULL,
  approved_at      TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_kyc_approval_customer ON customer_kyc_approval (customer_id, approved_at);
CREATE UNIQUE INDEX uq_kyc_one_per_level
  ON customer_kyc_approval (customer_id, approval_level)
  WHERE decision = 'APPROVED'; -- each level can have only one APPROVED row; rejections aren't constrained

```

The customer.kyc_status field is derived from the approval chain:

- No customer_kyc_approval rows → PENDING
- At least one APPROVED row but no is_final=true APPROVED row → LN_APPROVED where N is the highest approval_level present
- An APPROVED row with is_final=true → APPROVED
- Any decision=REJECTED row → REJECTED (rejections at any level terminate the chain)

The kyc_l1_approver_id, kyc_l1_approved_at, kyc_final_approver_id, kyc_final_approved_at columns are removed from customer (was in §2.1; updated below) — they're now derived from customer_kyc_approval. The kyc_status field on customer is maintained as a denormalized fast-read field, updated by trigger on insert into customer_kyc_approval.

5.3 KYC documents

Per workshop, KYC docs are captured by the field agent during the door-to-door visit. v1.0 stores **full structured references** including a path so any service can retrieve the document for retry, OCR, audit, etc. — per the SOPHIX file-storage foundation.

Documents are stored as rows in a dedicated table (not as JSONB on customer), giving operations a queryable inventory:

```

CREATE TABLE customer_kyc_document (
  document_id      TEXT PRIMARY KEY,          -- 'kycdoc_01HX...' ULID; this is the operational ID
  customer_id      TEXT NOT NULL REFERENCES customer(customer_id),
  document_type    TEXT NOT NULL,             -- 'NATIONAL_ID_FRONT' | 'NATIONAL_ID_BACK' | 'PASSPORT' | 'BUSIN
  document_name    TEXT NOT NULL,             -- human-readable name; e.g., 'Jane Mwangi ID Front'
  storage_path     TEXT NOT NULL,             -- foundation file-storage path; e.g., 's3://sophix-files-wik/kyc
  mime_type       TEXT NOT NULL,             -- 'image/jpeg' | 'application/pdf'
  size_bytes       INTEGER NOT NULL,
  content_hash     TEXT NOT NULL,             -- SHA-256 hex for integrity verification + dedup detection
  captured_by     TEXT NOT NULL,             -- 'agent_pwa:franchise_KE-NRB-04'
  captured_at      TIMESTAMP NOT NULL,
  verification_method TEXT NOT NULL,          -- 'VISUAL' | 'IPRS_OCR_FUTURE' | 'MANUAL_REVIEW'
  retention_class  TEXT NOT NULL DEFAULT 'P10Y', -- per FOUNDATION_DB retention discipline
  superseded_by_id TEXT NULL,                 -- if document was re-captured
  superseded_at    TIMESTAMP NULL,
  created_at      TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_kydoc_customer ON customer_kyc_document (customer_id);

```

```
CREATE INDEX idx_kycdoc_path      ON customer_kyc_document (storage_path);
CREATE INDEX idx_kycdoc_hash     ON customer_kyc_document (content_hash);
```

`storage_path` follows the SOPHIX foundation file-storage convention (path includes operator partition, kyc subfolder, year/month for retention, customer ID, document type). Any service can retrieve the bytes by calling the foundation file-storage service with the path. ILM-CFG-01 does not hold the bytes; it holds the structured reference + integrity hash.

`kyc_documents_jsonb` column is removed from `customer` (was in §2.1) — KYC documents are now first-class rows in `customer_kyc_document`.

Sample row:

document_id	customer_id	document_type	document_name	storage_path	mime_type	size_bytes	captured_by
kycdoc_01HX_JANE_ID_FRONT	cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6	NATIONAL_ID_FRONT	Jane Mwangi ID Front	s3://sophix-files-wik/kyc/2026/02/cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6/national_id_front.jpg	image/jpeg	423012	agent_pwa:franchise_KE-NRB-04
kycdoc_01HX_JANE_POA	cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6	PROOF_OF_ADDRESS	Jane Mwangi POA	s3://sophix-files-wik/kyc/2026/02/cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6/proof_of_address.pdf	application/pdf	198765	agent_pwa:franchise_KE-NRB-04

5.4 KYC approval API — single endpoint

One endpoint accepts approvals at any level. The request carries `approvalLevel` (integer) + `approvalLevelName` (display label) + `isFinal` (boolean — when true, KYC status advances to APPROVED on this submission). This is the single approval entry point for all operators and all approval-chain depths.

```
POST /api/customers/{customer_id}/kyc/approvals HTTP/1.1
Authorization: Bearer <bo-user-token>
```

```
{
  "approvalLevel": 1,
  "approvalLevelName": "L1_SUPERVISOR",
  "decision": "APPROVED",
  "isFinal": false,
  "approverId": "supervisor_franchise_KE-NRB-04",
  "approverRole": "L1_SUPERVISOR",
  "comments": "ID verified visually; POA matches Karen postal code"
}
```

→ 200 OK

```
{
  "approvalId": "kycappr_01HX_JANE_L1",
  "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
  "kycStatus": "L1_APPROVED",
  "approvedAt": "2026-02-17T08:15:00Z"
}
```

Final approval — same endpoint, `isFinal=true`:

```
POST /api/customers/{customer_id}/kyc/approvals
```

```
{
  "approvalLevel": 2,
  "approvalLevelName": "TEAM_LEADER",
  "decision": "APPROVED",
  "isFinal": true,
  "approverId": "team_leader_03",
  "approverRole": "TEAM_LEADER",
  "comments": "Approved"
}
```

→ 200 OK

```
{
  "approvalId": "kycappr_01HX_JANE_FINAL",
  "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
  "kycStatus": "APPROVED",
  "approvedAt": "2026-02-17T09:30:00Z"
}
```

Rejection — same endpoint, `decision=REJECTED`:

```
POST /api/customers/{customer_id}/kyc/approvals
```



```
{
  "approvalLevel": 1,
  "approvalLevelName": "L1_SUPERVISOR",
  "decision": "REJECTED",
  "isFinal": true,
  "approverId": "supervisor_franchise_KE-NRB-04",
  "approverRole": "L1_SUPERVISOR",
  "comments": "ID does not match Customer photo"
}
```

REJECTED at any level closes the chain (`isFinal=true` forced server-side for rejections). `customer.kyc_status` advances to REJECTED.

The two-level enforcement is at the API layer: calling with `approvalLevel=2`, `isFinal=true` on a customer that doesn't have an `approvalLevel=1`, `decision=APPROVED` row returns 409 `KYC_PREREQUISITE_LEVELS_MISSING`.

5.5 KYC rules

Rule	Description
R-ILM-K-1	<code>customer.kyc_status</code> is derived from <code>customer_kyc_approval</code> rows. Maintained by trigger on insert. Possible values: PENDING (no rows), LN_APPROVED (highest non-final approved level), APPROVED (final approval row present), REJECTED (any rejection row), WITHDRAWN (admin-set).
R-ILM-K-2	An approval at level N requires at least one APPROVED row at level N-1 (when $N > 1$). Enforced server-side; 409 <code>KYC_PREREQUISITE_LEVELS_MISSING</code> on violation. v1.0 KE WIK uses only levels 1 (L1_SUPERVISOR) and 2 (TEAM_LEADER, <code>isFinal=true</code>).
R-ILM-K-3	The approving user must have a FOUNDATION_AUTH role that matches the approval level. Mapping per operator config: KE L1 → role kenya - l1 - kyc; KE Final → role kenya - kyc - leaders. Future operators register their own role mapping.
R-ILM-K-4	Each approval emits a Kafka event. LN_APPROVED for non-final approvals (where $N = \text{approval_level}$); KYC_APPROVED when <code>isFinal=true</code> ; KYC_REJECTED for rejections. FUL-02 STEP-VALIDATE consumes only the final KYC_APPROVED for its CUSTOMER_ELIGIBLE check.
R-ILM-K-5	KYC documents are stored as first-class rows in <code>customer_kyc_document</code> with full path/mime/hash; ILM-CFG-01 holds references, not bytes. The bytes live in the SOPHIX foundation file-storage system.
R-ILM-K-6	<code>customer_kyc_document.content_hash</code> (SHA-256) supports integrity verification on retrieval AND duplicate-document detection within the same customer (e.g., re-captured ID) AND across customers (fraud detection — same passport image used for multiple customers).
R-ILM-K-7	KYC approval emits <code>CustomerAccountAvailableForActivation</code> event for each non-superseded account belonging to the customer when <code>kyc_status</code> reaches APPROVED. This is the trigger for FUL-02 STEP-VALIDATE's CUSTOMER_ELIGIBLE clearance.
R-ILM-K-8	Document supersession (<code>super_seded_by_id</code>) preserves the original document row + audit trail; never hard-deletes. GDPR purge is via the retention-class job, not via document update.

6. Customer search

Per workshop: "Search keys — Account Number · Address · Contact Methods · Name · Identification · Equipment ID · Drop Tag · PPV Lookup · Last Accessed Homes · with result columns Address · Last Name · First Name · M St · E Stat · City · Pending WO · Type (RES / COM / DEL) · Account Name."

v1.0 covers six search keys; Equipment ID and Drop Tag are covered by WO module / future Inventory and out of ILM-CFG-01 scope.

6.1 Search API

```
GET /api/customers/search?key=msisdn&value=%2B254712345678&limit=10
```

Supported keys:

- `account_number` — exact match on the internal SOPHIX-generated key
- `payment_account_number` — exact match on the gateway-callback reference; used by BIL-01 / BIL-05 when resolving an incoming payment callback to an account

- `msisdn` — exact match against `customer_contact_method.value` where `channel=MSISDN`
- `email` — exact match against `contact_method.value` where `channel=EMAIL`
- `name` — fuzzy trigram-similarity match on `customer.name`
- `identification_number` — exact match against either `identification_number_1` OR `identification_number_2` (e.g., searching by ECOWAS card-ID hits a customer whose `_1` is the card-ID; searching by national-ID hits the same customer whose `_2` is the national-ID — single search across both slots)
- `address` — fuzzy match on `customer_account.service_address`

Response shape mirrors the workshop's result columns:

```
{
  "results": [
    {
      "customer_id": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
      "account_id": "acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7",
      "account_number": "002-0598570K",
      "address": "Karen Hardy, Karen, Nairobi",
      "last_name": "Mwangi",
      "first_name": "Jane",
      "main_status": "ACTIVE",
      "sub_status": "active",
      "city": "Nairobi",
      "pending_wo_count": 0,
      "customer_type": "RES",
      "account_name": "Mwangi, Jane"
    }
  ],
  "totalMatches": 1
}
```

The `pending_wo_count` is computed by a cross-module read against WO module (GET `/api/work-orders/count?customerId=...&statusNotIn=COMPLETED,CANCELLED`). For search performance, this is materialized on the customer record via Kafka consumption of WO module events (consumer maintains the count). Stale data tolerance: 60s; cache invalidation on `WorkOrderCreated` / `WorkOrderFinalized` / `WorkOrderCancelled`.

6.2 Search rules

Rule	Description
R-ILM-SR-1	Search by <code>msisdn</code> MUST match the primary OR alt phone of any <code>contact_method</code> — workshop pattern is "search by primary phone returns the customer's accounts". v1.0 expands this to any phone or MSISDN-typed contact.
R-ILM-SR-2	Fuzzy name search uses Postgres trigram similarity with a default threshold of 0.3. Returns results above the threshold ranked by similarity descending.
R-ILM-SR-3	Search is scoped to the requesting BO user's operator via the auth context. Cross-operator search is reserved for super-admin roles (FOUNDATION_AUTH role <code>sophix-superadmin</code>).
R-ILM-SR-4	Search results are audit-logged per workshop's audit-on-read principle (every BO-agent search is recorded with <code>who</code> · <code>when</code> · <code>which customer</code> · <code>which filter</code>). The audit log is the <code>bo_audit_log</code> shared infrastructure table.
R-ILM-SR-5	Search is paginated; default <code>limit=20</code> , max <code>limit=100</code> . Pagination via offset cursor returned in response.

7. REST API

The module surface for other modules + BO UIs to consume. All endpoints scoped to the requesting BO user's operator via auth context. v1.0 ships these endpoints; future versions can add per-operator variants.

7.1 Customer lifecycle

Endpoint	Purpose
POST <code>/api/customers</code>	Create customer (RES or COM). Returns <code>customer_id</code> . Triggers <code>CustomerCreated</code> event.

Endpoint	Purpose
GET /api/customers/{id}	Fetch full customer with related accounts + contacts + most-recent notes. The 360° "single pane of glass" call returns ILM-owned data; subscription details are fetched by the caller from SUB-LM-01 using each account's customer_id + homepass_id.
PATCH /api/customers/{id}	Update mutable customer fields (name, email, preferred_language). Service class + attention banner + flags + status are per-account — see /api/customer-accounts/{id} endpoints. Triggers CustomerUpdated event.
POST /api/customers/{id}/kyc/l1-approve	L1 KYC approval (§5.3).
POST /api/customers/{id}/kyc/final-approve	Final KYC approval (§5.3).
POST /api/customers/{id}/kyc/reject	KYC rejection (at any approval level).
GET /api/customers/search	Search (§6.1).
DELETE /api/customers/{id}	Soft-delete (GDPR forget me sub-status). Hard-delete is reserved for post-retention purges by privileged jobs only.

7.2 Account lifecycle

Endpoint	Purpose
POST /api/customer-accounts	Create account (under existing customer). Returns account_id + account_number. subscription_id NULL initially. Triggers CustomerAccountCreated event.
GET /api/customer-accounts/{id}	Fetch account with status, sub-status, address, subscription_id. Subscription detail itself is obtained by direct call to SUB-LM-01 using the FK.
GET /api/customer-accounts?customerId=...	List all accounts for a customer.
PATCH /api/customer-accounts/{id}	Update mutable fields (service_address, account_manager_id, service_class_*, attention_banner). Flag-management endpoints are under §3.5 (account flag system).
PUT /api/customer-accounts/{id}/sub-status	Change sub-status. Auto-derives main status (ACTIVE vs INACTIVE) from cloned_from in catalog. Triggers CustomerAccountStatusChanged event.
PUT /api/customer-accounts/{id}/subscription	Bind a subscription to the account (sets subscription_id). Enforces 1:1 — rejects if the subscription_id is already bound to another account. Triggers CustomerAccountSubscriptionLinked event.
DELETE /api/customer-accounts/{id}/subscription	Unbind the subscription (sets subscription_id to NULL). Used during MACD terminate-and-recreate flows; followed by a fresh PUT to bind the new subscription. Triggers CustomerAccountSubscriptionUnlinked event.

Sample sub-status change:

```

PUT /api/customer-accounts/acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7/sub-status
{
  "newSubStatus":      "complementary discount",
  "changeReason":      "Outage compensation for April 2026 incidents",
  "approvalReference": "approval_ticket_TKT-2026-0419-001",
  "changedBy":         "bo_user_billing_05",
  "changedByRole":     "BILLING_TEAM",
  "requestChannel":    "EMAIL"
}

→ 200 OK
{
  "accountId": "acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7",
  "status":    "ACTIVE",
  "subStatus": "complementary discount",
  "changedAt": "2026-05-18T11:00:00Z"
}

```

Sample subscription bind (called by FUL-02 STEP-SUBSCRIPTION after SUB-LM-01 creates the subscription record):

```

PUT /api/customer-accounts/acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7/subscription
{
  "subscriptionId": "subs_01HX_JANE_TRIPLE_PLAY",
  "linkedBy":       "system:ful-02",
  "linkedByRole":   "SYSTEM"
}

→ 200 OK
{
  "accountId":      "acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7",
  "subscriptionId": "subs_01HX_JANE_TRIPLE_PLAY",
}

```

```

    "linkedAt":      "2026-05-18T11:30:00Z"
  }

```

Errors:

Status	When
409 SUBSCRIPTION_ALREADY_LINKED	The subscription_id is already bound to another account (1:1 violation)
409 ACCOUNT_ALREADY_LINKED	This account already has a non-NULL subscription_id; caller must DELETE first
404 ACCOUNT_NOT_FOUND	The account_id doesn't exist
422 OPERATOR_MISMATCH	The Subscription's operator differs from the Account's operator

7.3 Subscription data — owned by SUB-LM-01

ILM-CFG-01 does NOT expose subscription detail endpoints. Callers needing subscription data follow the FK directly. A typical "view all services for this customer" UI call sequence:

1. GET /api/customers/{id} → ILM-CFG-01 returns the customer + accounts list (with subscription_id on each account).
2. For each account with a non-NULL subscription_id: GET /api/subscriptions/{subscription_id} → SUB-LM-01 returns the subscription detail (package, status, workflow data).

The 1:1 FK pattern means each lookup is O(1) — no WHERE customer_id = ... AND homepass_id = ... searches needed.

7.4 Contacts

Endpoint	Purpose
POST /api/customers/{id}/contact-methods	Add a contact method. Enforces R-ILM-C-1 if attempting to add another primary MSISDN — supersedes the existing one.
GET /api/customers/{id}/contact-methods	List non-superseded contact methods for the customer.
PATCH /api/contact-methods/{id}	Update label, verified flag, is_primary, is_billing. Supersession of an MSISDN/primary contact creates a new row referencing this.
POST /api/contact-methods/{id}/verify	Mark a contact verified (OTP flow handled by future DD_NOT-01; v1.0 admin-mark-only).

7.5 Notes

Endpoint	Purpose
POST /api/customers/{id}/notes	Append a note. If is_attention=true, supersedes the existing attention banner.
GET /api/customers/{id}/notes?limit=20&kind=...	List notes (most-recent first).
PATCH /api/notes/{id}/supersede	Mark a note superseded by a new note id. The replacement note row must be passed.

7.6 Interactions

Endpoint	Purpose
POST /api/customers/{id}/interactions	Append structured interaction.
GET /api/customers/{id}/interactions?limit=20	List interactions (most-recent first).

7.7 Notification preferences

Endpoint	Purpose
GET /api/customer-accounts/{id}/notification-pref	Read account-scoped notification preferences. Consumed by DD_NOT-01 before sending customer-facing notifications scoped to a specific account.
PUT /api/customer-accounts/{id}/notification-pref	Update notification preferences for this account only.
GET /api/customers/{id}/notification-prefs	List notification preferences for ALL the customer's accounts (BO UI summary view).

7.8 Account flags

Endpoint	Purpose
GET /api/customer-accounts/{id}/flags	List all current flags on the account with values + evaluator + evaluated_at.
GET /api/customer-accounts/{id}/flags/{flag_code}	Read a specific flag.
PUT /api/customer-accounts/{id}/flags/{flag_code}	Manually set/update a flag value (only allowed for flags with catalog.evaluator='MANUAL'; rejected with 409 for Drools-evaluated flags).
DELETE /api/customer-accounts/{id}/flags/{flag_code}	Manually clear a flag (only allowed for MANUAL flags).
GET /api/customer-account-flag-catalog?operatorCode=WIK	List the operator's flag catalog. Used by BO UI to render flag-management forms.
GET /api/customer-accounts/search?key=flag&flagCode=NPD&value=true	Find accounts where a given flag has a given value. Used by BIL-04 dunning (flag=NPD, value=true) and Finance reports.

Sample manual flag set:

```
PUT /api/customer-accounts/acct_01HZP9Q8R7S6T5U4V3W2X1Y0Za/flags/FRAUD_SUSPECTED
{
  "valueBoolean":    true,
  "evaluatedBy":     "bo_user_fraud_team_02",
  "evaluatorKind":   "MANUAL",
  "evaluationContext": {
    "trigger":        "Customer reported card fraud 2026-04-12; account flagged pending investigation",
    "ticketRef":      "TKT-FRAUD-2026-0412-001"
  }
}

→ 200 OK
{
  "accountId":       "acct_01HZP9Q8R7S6T5U4V3W2X1Y0Za",
  "flagCode":        "FRAUD_SUSPECTED",
  "valueBoolean":    true,
  "evaluatedAt":     "2026-04-12T11:30:00Z"
}
```

7.9 Common patterns

All POST/PATCH/PUT endpoints:

- Accept an optional Idempotency-Key header. Idempotent on (operator_code, endpoint, idempotency_key). Repeated calls within 24h return the original 2xx response.
- Audit-log on success: bo_audit_log row with actor_id, action, entity_id, request_body_redacted, response_summary.
- Audit-log on failure: bo_audit_log row with actor_id, action, entity_id, error_code.
- Reject if the requesting user's operator does not match the entity's operator (cross-operator writes are reserved for sophix-superadmin).

8. Kafka events

ILM-CFG-01 publishes events on `sophix.customer.*` and consumes events from other modules to keep the customer state in sync.

8.1 Events published

Event type	Topic	Emitted when	Key consumers
CustomerCreated	<code>sophix.customer.created</code>	New customer row inserted	FUL-02 STEP-CAPTURE (acknowledge), CVM analytics
CustomerUpdated	<code>sophix.customer.updated</code>	customer row mutated	CRM-style downstream caches, CVM
CustomerKycL1Approved	<code>sophix.customer.kyc-l1-approved</code>	L1 KYC approval recorded	None v1.0 (audit)
CustomerKycApproved	<code>sophix.customer.kyc-approved</code>	Final KYC approval recorded	FUL-02 STEP-VALIDATE (CUSTOMER_ELIGIBLE check)
CustomerKycRejected	<code>sophix.customer.kyc-rejected</code>	KYC rejected at any level	FUL-02 STEP-CANCELLATION (if active order exists)

Event type	Topic	Emitted when	Key consumers
CustomerAccountCreated	sophix.customer.account-created	New account row inserted	RLM-CFG-01 (HomePass linkage), Finance
CustomerAccountUpdated	sophix.customer.account-updated	Account row mutated	CVM, BO dashboards
CustomerAccountStatusChanged	sophix.customer.account-status-changed	Account status or sub_status changed	FUL-03 / provisioning (when affects_provisioning=true), BIL-04 (pause-aware dunning), DD_NOT-01 (customer-visible status notifications), WO module (for VIP-aware Drools assignment)
CustomerAccountSubscriptionLinked	sophix.customer.account-subscription-linked	customer_account.subscription_id set from NULL to a Subscription ID (or rebound to a new one)	BO dashboards, CVM, Finance
CustomerAccountSubscriptionUnlinked	sophix.customer.account-subscription-unlinked	customer_account.subscription_id cleared to NULL	BO dashboards, CVM
CustomerAccountFlagChanged	sophix.customer.account-flag-changed	A flag value set, changed, or cleared on customer_account_flag	BIL-04 (dunning escalation logic), FUL-03 (provisioning gates), CVM, BO dashboards
CustomerAccountFlagCleared	sophix.customer.account-flag-cleared	A flag row deleted from customer_account_flag (e.g., manual clear, Drools rule no longer matches)	BIL-04, FUL-03, CVM
CustomerContactMethodChanged	sophix.customer.contact-method-changed	Contact method added, updated, or superseded	DD_NOT-01 (channel preferences), CRM caches
CustomerNoteAppended	sophix.customer.note-appended	New note row inserted	Future Ticketing module (correlate notes to tickets)
CustomerAccountAttentionBannerChanged	sophix.customer.account-attention-banner-changed	An account's effective attention banner changed	BO UIs (real-time banner refresh)
CustomerForgetMeRequested	sophix.customer.forget-me-requested	GDPR forget-me sub-status entered	Future data-retention purge job

8.2 Sample event — CustomerAccountStatusChanged

```
{
  "eventType": "CustomerAccountStatusChanged",
  "aggregateId": "acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7",
  "timestamp": "2026-05-18T11:00:00.234Z",
  "payload": {
    "accountId": "acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "previousStatus": "ACTIVE",
    "previousSubStatus": "active",
    "newStatus": "ACTIVE",
    "newSubStatus": "complementary discount",
    "changeReason": "Outage compensation for April 2026 incidents",
    "approvalReference": "approval_ticket_TKT-2026-0419-001",
    "changedBy": "bo_user_billing_05",
    "changedByRole": "BILLING_TEAM",
    "requestChannel": "EMAIL",
    "sourceEventRef": null,
    "affectsProvisioning": false,
    "customerVisible": true,
    "changedAt": "2026-05-18T11:00:00.000Z"
  }
}
```

8.3 Sample event — CustomerKycApproved

```
{
  "eventType": "CustomerKycApproved",
  "aggregateId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
  "timestamp": "2026-02-17T09:30:00.123Z",
  "payload": {
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "kycL1ApproverId": "supervisor_franchise_KE-NRB-04",
    "kycL1ApprovedAt": "2026-02-17T08:15:00Z",
    "kycFinalApproverId": "team_leader_03",
    "kycFinalApprovedAt": "2026-02-17T09:30:00Z",
    "kycDocumentRefs": ["idoc_01HX_JANE_ID_FRONT", "idoc_01HX_JANE_POA"]
  }
}
```

```
}
```

8.4 Sample event — CustomerAccountFlagChanged

```
{
  "eventType": "CustomerAccountFlagChanged",
  "aggregateId": "acct_01HZ_PETER_KIMANI_NPD_ACCT",
  "timestamp": "2026-05-20T02:00:14.567Z",
  "payload": {
    "accountId": "acct_01HZ_PETER_KIMANI_NPD_ACCT",
    "customerId": "cust_01HZ_PETER_KIMANI_NPD",
    "operatorCode": "WIK",
    "flagCode": "NPD",
    "previousValue": {"valueBoolean": false},
    "newValue": {"valueBoolean": true},
    "evaluatorKind": "DROOLS",
    "evaluatedBy": "system:flag-evaluator-wik-v1",
    "evaluationContext": {
      "lastPaymentDays": 47,
      "openInvoiceCount": 2,
      "totalOverdueAmount": 5200,
      "currency": "KES",
      "droolsRuleVersion": "flag-evaluator-wik-v1:npd-rule@2026.04.01"
    },
    "evaluatedAt": "2026-05-20T02:00:14.000Z",
    "affectsDunning": true,
    "affectsProvisioning": false,
    "customerVisible": false
  }
}
```

8.5 Events consumed

Event type	From module	What ILM-CFG-01 does
OrderActivated	FUL-02 STEP-ACTIVATION	Sets customer_account.status = ACTIVE, sub_status = active. Subscription status updates are owned by SUB-LM-01's activation workflow.
SubscriptionCreated	SUB-LM-01	Looks up the matching Account by (customer_id, homepass_id); if found and the Account's subscription_id IS NULL, binds the new subscription via subscription_id = <new>. Emits CustomerAccountSubscriptionLinked. If the Account already has a non-NULL subscription_id, the consumer fails the event (no auto-rebind in v1.0; rebinding requires explicit MACD workflow).
WorkOrderPhaseFinalized (Shifting flow)	WO module	Reads payload.accountSubStatusChange. If {to: INA, ...}: account → INACTIVE inactive pending customer. If {to: STF, ...}: account → ACTIVE staff account (or operator-equivalent).
DunningLevelEntered	BIL-04	If new level = SUSPENDED (level 3): account → INACTIVE inactive pending customer. If lower restriction level: don't change sub_status, just record on history with source_event_ref.
DunningRecovered	BIL-04	Account → ACTIVE active (revert to standard active state).
SubscriptionPaused	SUB-LM-01	No-op by default. Per-operator config can derive account sub_status from a paused subscription → e.g., seasonal disconnect; v1.0 KE has this OFF.
SubscriptionResumed	SUB-LM-01	No-op by default. Per above.
SubscriptionTerminated	SUB-LM-01	If the terminated Subscription is currently bound on an Account (i.e., customer_account.subscription_id matches), the consumer clears subscription_id = NULL and emits CustomerAccountSubscriptionUnlinked. Per-operator config can additionally transition the account to churned (cloned from INACTIVE); v1.0 KE keeps the status transition manual — BO agent transitions the account, no auto-derivation.

Event type	From module	What ILM-CFG-01 does
WorkOrderCreated / WorkOrderFinalized / WorkOrderCancelled	WO module	Maintains pending_wo_count materialized for search performance (R-ILM-SR-1 search context).

8.6 Consumer idempotency

Each consumer maintains a `customer_event_processed_idempotency` table keyed on (`operator_code`, `source_module`, `source_event_id`). Idempotency window: 30 days; older entries pruned.

9. Stub retirement

ILM-CFG-01 retires the `customer_stub` table that lives in FUL-02 FRAMEWORK §6.1 and is referenced by WO module FRAMEWORK §9.2.

9.1 Mapping `customer_stub` → ILM-CFG-01

The stub's 8-column schema maps to the full module:

Stub column	Maps to	Notes
<code>customer_id</code>	<code>customer.customer_id</code>	Same key.
<code>name</code>	<code>customer.name</code>	Direct migration.
<code>msisdn</code>	<code>customer_contact_method.value</code> where <code>channel=MSISDN</code> , <code>is_primary=TRUE</code>	Becomes a contact method row.
<code>email</code>	<code>customer_contact_method.value</code> where <code>channel=EMAIL</code> , <code>is_primary=TRUE</code> , <code>is_billing=TRUE</code>	Becomes a contact method row.
<code>id_document_ref</code>	<code>customer.kyc_documents_jsonb[0].document_ref</code>	Wrapped in the documents array.
<code>kyc_status</code>	<code>customer.kyc_status</code>	Direct migration. L1_APPROVED value preserved.
<code>operator_code</code>	<code>customer.operator_code</code>	Direct migration.
<code>created_at</code>	<code>customer.created_at</code>	Direct migration.

9.2 Migration steps

- Pre-deployment:** ILM-CFG-01 schema deployed alongside `customer_stub`. Both tables coexist during transition.
- Backfill:** One-time job migrates every `customer_stub` row to `customer` + `customer_contact_method` (MSISDN + email) + `customer_account` (one stub account placeholder per stub row, until the real account creation flow runs).
- Caller cutover:** FUL-02 (STEP-CAPTURE, STEP-VALIDATE) + WO module (BO UI customer-lookup, print PDF rendering) switch their GET `customer_stub` calls to GET `/api/customers/{id}` from ILM-CFG-01. Compatibility shim: ILM-CFG-01 serves the lookup at the same logical contract.
- Verification:** Run for 1 week with both systems updated dual-write style if any new customers are being created; reconcile daily.
- Removal:** Drop `customer_stub` table. Remove FUL-02 FRAMEWORK §6.1 and WO module FRAMEWORK §9.2 schema docs (replace with "retired — see DD_ILM-CFG-01" reference).

9.3 Customer creation flow change

In v1.0 with `customer_stub`, customer rows are pre-populated by the field agent's CRM tooling at sales-conversation time (per FUL-02 STEP-CAPTURE). After ILM-CFG-01 ships:

- The field agent's CRM tooling calls POST `/api/customers` directly to create the customer at sales-conversation time. The endpoint returns `customer_id` + initial `kyc_status=PENDING`.
- The agent also calls POST `/api/customer-accounts` to create the deployment account (without `homepass_id` yet; populated during install).
- KYC docs are captured via PATCH `/api/customers/{id}` with `kyc_documents_jsonb`.
- The L1 supervisor calls POST `/api/customers/{id}/kyc/l1-approve`; the Team Leader calls `final-approve`.

- When KYC = APPROVED, the CustomerKycApproved event fires, FUL-02 STEP-VALIDATE consumes it for CUSTOMER_ELIGIBLE clearance.

The pre-V3 redeploy plan's "dummy-account-from-pool RPA" pattern is fully retired — customer creation is now first-class, on-demand, API-driven.

10. Workshop sources

The Customer Management workshop (deep-dive 01, 2026-04-27 / 2026-04-28 at Zuku offices) drives this DD's scope. Specific feature areas covered:

- **Customer Account Creation** — door-to-door capture on Field Agent PWA; 3-tier CAS model; "1 account = 1 deployment" rule; account number as the only operational key; V3 first-class customer entity replacing dummy-account-from-pool.
- **Customer Profile Management** — 360° single pane of glass; RBAC-gated sections; read-only profile with mutations through dedicated UCs; service-class badge informational only.
- **KYC Management** — door-to-door + L1 (regional/franchise supervisor) + final approval (Wananchi Team Leader); accepted IDs (National ID, Passport, Business Registration); planned IPRS integration as future enhancement.
- **Package Assignment** — services derived from package; WO creation rule per equipment-delta; wallet funding gate for contractor WOs.
- **MACD Operations** — Move/Add/Change/Disconnect flows (these are handled by WO module + SUB-LM-01 + FUL-02; ILM-CFG-01 surfaces the customer state, doesn't own MACD orchestration).
- **Account Status Management** — main statuses (INACTIVE / ACTIVE / SUSPENDED / CHURN / TERMINATED + sub-statuses + service class). Workshop ambiguity: SUSPENDED / CHURN / TERMINATED are presented as additional main statuses in one section and as sub-statuses cloned from INACTIVE in another section. V3 design follows the "Customer Preferences" workshop view: **only INACTIVE and ACTIVE as main statuses, everything else as sub-statuses** (workshop quote: "Base statuses are only INACTIVE and ACTIVE; everything operational below is a sub-status"). This collapses SUSPENDED/CHURN/TERMINATED into the sub-status registry. Specifically: churned (in catalog), and the dunning-driven inactives (inactive pending customer, hold). TERMINATED is a subscription-level state in SUB-LM-01 (subscription.status_code = TERMINATED), not an ILM account state.
- **Customer Interaction History** — structured Last Note per workshop (Agent name · Call reason · Findings · Resolution · VOC).
- **Customer Notes** — Attention banner + Last Note.
- **Customer Contact Management** — primary phone (MSISDN) required; multiple channels.
- **Customer Segmentation** — service class flags (VIP / Platinum / Gold / STAFF) informational only.
- **Customer Search** — by Account #, Address, Contact Methods, Name, Identification, Equipment ID, Drop Tag, PPV Lookup, Last Accessed Homes; result columns per workshop.
- **Customer Document Management** — KYC/contract held in Agent App; install photos with contractor's FTP; future unified DM is "good-to-have" — out of v1.0 scope.
- **Customer Notifications** — preference held per-customer; channel rule "Email if available, otherwise SMS".
- **Customer Preferences** — implemented as sub-statuses (cloned-from-main pattern); v1.0 uses the same sub-status registry.
- **Transaction History** — read from BIL modules; not ILM-CFG-01-owned.

Out of v1.0 scope per workshop or design decision:

- **Multi-location Account Support** — workshop says "not used in Kenya / Wananchi"; in Kenya each location = separate Account. The natural multi-Account-per-Customer model handles this.
- **Customer Bulk Import** — workshop "not used in Kenya".
- **Customer Credit Limit** — workshop "not used in Kenya / Wananchi — postpaid feature".
- **Customer Risk Profile** — workshop "not used in Kenya"; the workshop's NPD boolean flag is supported as one entry in v1.0's Account flag system (§3.5). Future flag types (churn risk, fraud suspected) are operator-extensible additions to the catalog, not new schema work.
- **Customer Document Management** — future unified DM module.

11. Cross-DD coordination

11.1 Module owner

This module is owned by the Foundations team (Group I) alongside other foundational modules (PLM-CFG-01, SIP-01, RLM-CFG-01). It's a peer-of dependency for FUL-* and WO-*; an authoritative source for everything customer-related.

11.2 Upstream callers

Module	What they call	When
Field Agent PWA	POST /api/customers, POST /api/customer-accounts, POST /api/customers/{id}/kyc/l1-approve, PATCH /api/customers/{id} (with KYC docs)	Door-to-door sales + onboarding
BO UI (CSR Console)	Most read endpoints + PUT .../sub-status, POST .../notes, POST .../interactions	Customer service operations
BO UI (Backoffice Dispatcher)	Search + customer profile read + WO links	Operational lookups
FUL-02 STEP-CAPTURE	GET /api/customers/{id}	Validate the customer exists before order capture
FUL-02 STEP-VALIDATE	GET /api/customers/{id} + consume CustomerKycApproved event	CUSTOMER_ELIGIBLE check
WO module BO UI	GET /api/customers/{id}	Customer info on WO display + print PDF
Future Ticketing module	GET /api/customers/{id}, POST .../notes, POST .../interactions	Ticket-side customer lookup

11.3 Downstream consumers

Module	What they consume	What they do
FUL-02 STEP-VALIDATE	CustomerKycApproved event	Gate the CUSTOMER_ELIGIBLE check
FUL-03 / provisioning	CustomerAccountStatusChanged (when affects_provisioning=true)	Apply network-level state changes (disable service when account goes INACTIVE, enable when ACTIVE)
BIL-04 dunning	CustomerAccountStatusChanged	Pause-aware dunning: dunning escalation doesn't fire against accounts in seasonal disconnect, etc.
DD_NOT-01	CustomerAccountStatusChanged (when customer_visible=true), CustomerContactMethodChanged, CustomerKycApproved/Rejected	Customer-facing notifications (e.g., "Welcome — KYC approved")
CVM analytics	All events on <code>sophix.customer.*</code>	Real-time customer-state-change analytics
WO module Drools	CustomerAccountStatusChanged (cached)	VIP-aware contractor assignment

11.4 Sibling modules (peer dependencies)

Module	Direction	Contract
SUB-LM-01 (Subscription Lifecycle)	ILM ↔ SUB-LM	SUB-LM-01 owns the Subscription entity. Both modules share <code>customer_id</code> and <code>homepass_id</code> references; SUB-LM-01 looks up customer info from ILM-CFG-01 at activation. ILM-CFG-01 makes no FK to SUB-LM-01.
PLM-CFG-01 (Service / Product)	ILM ← PLM	Reads <code>package_ref</code> for display where applicable; doesn't write to PLM
SIP-01 (Package)	ILM ← SIP	(read indirectly via SUB-LM-01)
RLM-CFG-01 (HomePass)	ILM ↔ RLM	<code>customer_account.homepass_id</code> is FK to RLM; RLM-CFG-01 consumes <code>CustomerAccountCreated</code> for HomePass linkage updates

Module	Direction	Contract
BIL-* modules	ILM ↔ BIL	Invoice-per-Account scope. Because Account ↔ Subscription is 1:1, BIL-02-GEN-01 generates one Invoice per Account per billing cycle, containing line items for the Services delivered by that Account's single Subscription. ILM-CFG-01 surfaces the Account-to-Invoice join on the "single pane of glass" by calling BIL-02-GEN-01 with accountId (or subscriptionId, which resolves 1:1). Payment reconciliation. BIL-01 + BIL-05 read customer_account.payment_account_number to resolve incoming payment-gateway callbacks to an Account via GET /api/customer-accounts/search?key=payment_account_number&value=<ref>, then apply the payment to that Account's open Invoice(s). Mutability. When the operator's payment-reference scheme changes (gateway migration, format change), Finance updates payment_account_number via the PATCH endpoint; BIL modules pick up the new reference on the next callback. Dunning + flags. BIL-04 dunning state machine operates per-Account — an Account whose Invoice goes unpaid is dunned independently. BIL-04 also reads Account flags (NPD, PAYMENT_DELAY_FREQUENT, etc.) via GET /api/customer-accounts/{id}/flags or via the flag-changed Kafka event stream, and uses them to decide dunning escalation policy.

11.5 Pre-existing stubs retired

Stub	Owner module	Status
customer_stub	FUL-02 FRAMEWORK §6.1, referenced by WO module FRAMEWORK §9.2	Retired by this DD (§9)

11.6 Dependencies (build order)

- PLM-CFG-01** and **SIP-01** must be shipped first — referenced by SUB-LM-01 (subscriptions for an account are queried from SUB-LM-01, not from ILM).
- RLM-CFG-01** must be shipped — customer_account.homepass_id references HomePass records.
- FOUNDATION_AUTH** must be configured — roles kenya-l1-kyc, kenya-kyc-leaders, sophix-superadmin are required.
- Kafka infrastructure** must be in place — events listed in §8.

ILM-CFG-01 can ship before WO module or FUL-02; the consumers cut over to ILM-CFG-01 from customer_stub once it's live.

12. Runtime walkthrough — narrative

One scenario: a new customer signs up door-to-door, KYC clears, account is activated; later, the customer goes into seasonal disconnect for travel, then comes back active. Shows the module's end-to-end lifecycle in customer terms.

12.1 Door-to-door capture

Day 0 — Sales agent at Jane Mwangi's door. Jane wants to sign up for GPON 50Mbps + VOIP for her Karen apartment. Agent opens Field Agent PWA, captures Jane's details.

Step 1 — Create the customer. PWA calls:

```
POST /api/customers HTTP/1.1
Authorization: Bearer <field-agent-token>
```

```
{
  "operatorCode":    "WIK",
  "type":           "RES",
  "name":           "Jane Mwangi",
  "identificationType": "NATIONAL_ID",
  "identificationNumber": "28471929",
  "dateOfBirth":    "1991-04-22",
}
```

```

    "primaryMsisdn":      "+254712345678",
    "email":              "jane.mwangi@example.com",
    "preferredLanguage":  "en"
  }
}

→ 201 Created
{
  "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
  "kycStatus":  "PENDING",
  "createdAt":  "2026-02-16T14:30:00Z"
}

```

Module side: insert into customer, insert MSISDN + email rows into customer_contact_method. Emit CustomerCreated + CustomerContactMethodChanged × 2.

Step 2 — Capture KYC docs. PWA calls:

```

PATCH /api/customers/cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6
{
  "kycDocumentsJsonb": {
    "documents": [
      {"documentType": "NATIONAL_ID", "documentRef": "idoc_01HX_JANE_ID_FRONT", "capturedAt": "2026-02-16T14:32:00Z", "capturedBy": "jane.mwangi@example.com"},
      {"documentType": "PROOF_OF_ADDRESS", "documentRef": "idoc_01HX_JANE_POA", "capturedAt": "2026-02-16T14:33:00Z", "capturedBy": "jane.mwangi@example.com"}
    ]
  }
}

→ 200 OK

```

Module side: update customer.kyc_documents_jsonb, emit CustomerUpdated.

Step 3 — Create the account. Customer's deployment address is Karen Hardy:

```

POST /api/customer-accounts
{
  "customerId":      "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
  "operatorCode":    "WIK",
  "serviceAddress":  "Karen Hardy, Karen, Nairobi",
  "homepassId":      "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
  "franchiseCode":   "KE-NRB-FRANCHISE-04",
  "status":          "INACTIVE",
  "subStatus":       "inactive pending customer"
}

→ 201 Created
{
  "accountId":       "acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7",
  "accountNumber":   "002-0598570K",
  "status":          "INACTIVE",
  "subStatus":       "inactive pending customer"
}

```

Module side: insert into customer_account + customer_account_status_history. Emit CustomerAccountCreated + CustomerAccountStatusChanged (with affects_provisioning=true → FUL-03 sees no-op since INACTIVE means nothing to provision yet).

12.2 KYC approval

Day 0 evening — L1 supervisor reviews. The franchise supervisor opens the BO UI, sees the pending customer queue, reviews Jane's docs:

```

POST /api/customers/cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6/kyc/l1-approve
{
  "approverId":      "supervisor_franchise_KE-NRB-04",
  "approverRole":    "L1_SUPERVISOR",
  "comments":        "ID verified visually; POA matches Karen postal code"
}

→ 200 OK { "kycStatus": "L1_APPROVED" }

```

Module side: update customer.kyc_status = L1_APPROVED, kyc_l1_approver_id, kyc_l1_approved_at. Emit CustomerKycL1Approved.

Day 1 morning — Wananchi Team Leader gives final approval:

```

POST /api/customers/cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6/kyc/final-approve
{
  "approverId":      "team_leader_03",
  "approverRole":    "TEAM_LEADER",
  "comments":        "Approved"
}

```

```
→ 200 OK { "kycStatus": "APPROVED" }
```

Module side: update `kyc_status = APPROVED`, `kyc_final_approver_id`, `kyc_final_approved_at`. Emit `CustomerKycApproved`.

FUL-02 STEP-VALIDATE has been parked waiting for this event (per its `await - kyc - approval` user task or via Kafka message correlation). It consumes the event, marks `CUSTOMER_ELIGIBLE` clear, the order continues to payment, install, activation.

12.3 Activation

Day 2 — Order activates. FUL-02 STEP-ACTIVATION completes after install. It emits `OrderActivated`. ILM-CFG-01 consumes:

1. Resolve customer + account from the event payload.
2. Look up the current sub-status for the account: `inactive pending customer`.
3. Determine new sub-status from the catalog default for "post-activation": `active`.
4. Update `customer_account.status = ACTIVE`, `sub_status = active`, `sub_status_changed_at = NOW()`.
5. Write `customer_account_status_history` row.
6. Emit `CustomerAccountStatusChanged` (with `affects_provisioning=true` because the sub-status catalog has it set for `active`).

Jane's Subscription in SUB-LM-01 is activated by SUB-LM-01's own activation workflow (SUB-WF-ACTIVATE-01) — independent of ILM-CFG-01's account-status side-effect. That Subscription carries a multi-Service Package (Internet 50M GPON + VOIP Hosted in one Package); both Services are provisioned as part of the same Subscription activation.

FUL-03 / provisioning consumes the `affectsProvisioning=true` event and applies the network-level activation (already done by SUB-WF-ACTIVATE-01 upstream — the event here is more an audit confirmation than the actual trigger).

12.4 Seasonal disconnect

Six months later — Jane emails BO: traveling overseas for 2 months, asks to suspend. BO agent enters the request in the BO UI:

```
PUT /api/customer-accounts/acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7/sub-status
{
  "newSubStatus":      "seasonal disconnect",
  "changeReason":      "Customer travel – Sept 1 to Nov 1; email request received 2026-08-25",
  "changedBy":         "bo_user_csr_07",
  "changedByRole":     "CSR",
  "requestChannel":     "EMAIL"
}

→ 200 OK
{
  "accountId": "acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7",
  "status":    "ACTIVE",
  "subStatus": "seasonal disconnect"
}
```

Module side:

1. Catalog lookup: `seasonal disconnect` is cloned from `ACTIVE`, `requires_approval=false`, `affects_provisioning=true`.
2. Update `customer_account.sub_status = seasonal disconnect`. (Status stays `ACTIVE` per the cloned-from rule.)
3. Write `customer_account_status_history` row.
4. Emit `CustomerAccountStatusChanged` with `previousSubStatus=active`, `newSubStatus=seasonal disconnect`, `affectsProvisioning=true`, `customerVisible=true`.

FUL-03 / provisioning consumes the event, suspends the network-level service (this is functionally similar to dunning suspension but with a different business meaning). DD_NOT-01 consumes the event (`customer_visible=true`), sends Jane a confirmation SMS: "Your service is paused per your request until further notice. Service will resume on request."

12.5 Return to active

Two months later — Jane is back, emails BO to reactivate:

```
PUT /api/customer-accounts/acct_01HX3K9M2P5Q8R1T4W7Y0Z3N7/sub-status
{
  "newSubStatus": "active",
  "changeReason": "Customer back from travel; email request 2026-11-02",
  "changedBy": "bo_user_csr_07",
  "changedByRole": "CSR",
  "requestChannel": "EMAIL"
}

→ 200 OK
```

Module side: account → ACTIVE active. Event emitted. FUL-03 / provisioning consumes and re-enables. DD_NOT-01 sends "Welcome back — service resumed" SMS.

12.6 Lifecycle summary

Within ILM-CFG-01, Jane's record now shows:

- Customer row: ACTIVE, KYC APPROVED, no attention banner.
- 1 account row in Karen, ACTIVE sub_status=active, no service_class flag.
- Status history: 4 rows (initial inactive-pending → ACTIVATED → seasonal disconnect → ACTIVE).
- Account flags (evaluated nightly by flag-evaluator-wik-v1): NPD=false, CHURN_RISK=12 (low), PAYMENT_DELAY_FREQUENT=false, HIGH_VALUE=false, LOYALTY_TIER=BRONZE (8 months tenure).
- Account notification preferences: email-primary, SMS fallback, all topic categories opted-in.
- Contact methods: 2 rows (MSISDN + email, both primary).
- Notes: 0 attention notes, ~3 free-text CSR notes from interactions.
- KYC: 2 approval rows (L1 + Final), 2 document rows (ID front + POA).

Subscription data lives in SUB-LM-01 — surfaced in BO UI by following the subscription_id FK from the Account: GET /api/subscriptions/subs_01HX_JANE_TRIPLE_PLAY returns Jane's single Subscription record. That Subscription's Package contains two Services (Internet 50M GPON, VOIP Hosted), both ACTIVE.

Total elapsed: 8 months across her customer journey. Module surface is consistent throughout.

End of DD_ILM-CFG-01-Customer_Service-v1.0.